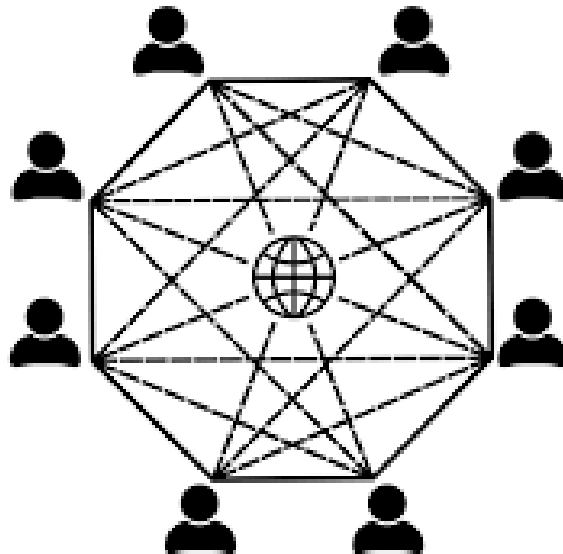
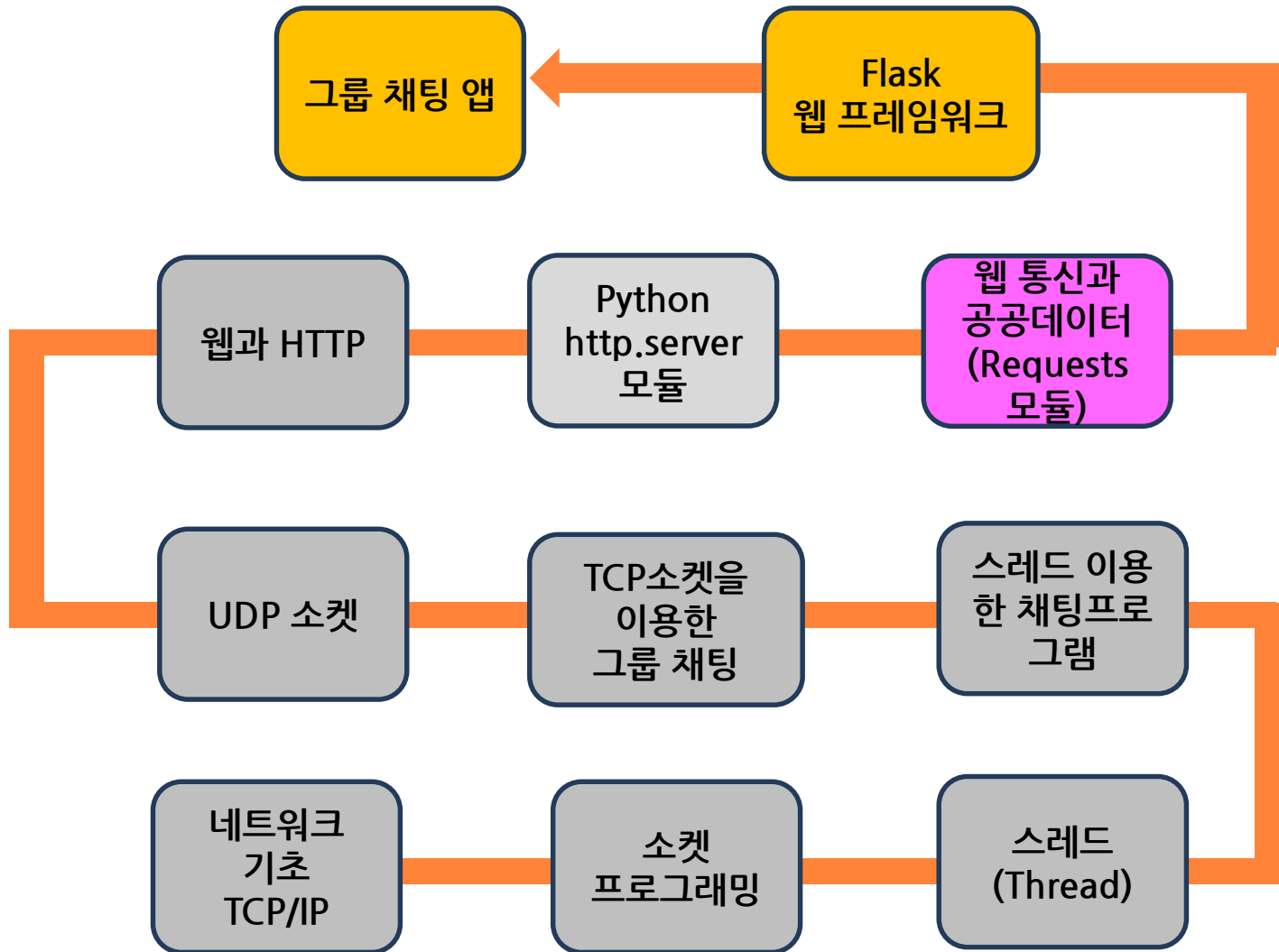


네트워크설계

11주차 - 공공데이터포털 API 이용하기



11주차는 OpenAPI를 이용한 공공데이터 활용



앱 제작 실습 병행

10주차 정리. http.server 모듈 개요

■ Python http.server 모듈

http.server — HTTP servers



- 파이썬으로 HTTP 서버를 구현하기 위한 기본 모듈

http.server는 별도의 외부 라이브러리 없이 파이썬 만으로 간단한 웹 서버를 쉽게 구축할 수 있게 해주는 내장 모듈

- 웹서버를 만드는데 필요한 라이브러리를 http.server 모듈에서 정의

- 핵심 클래스

1) **HTTPServer 클래스**: 웹 서버 객체를 만들기 위한 클래스

- 객체 생성시 서버 주소정보(IP와 PORT)와 핸들러 클래스 반드시 필요

2) **핸들러 클래스** : 어떤 요청이 들어올 때 어떻게 응답할지 정의

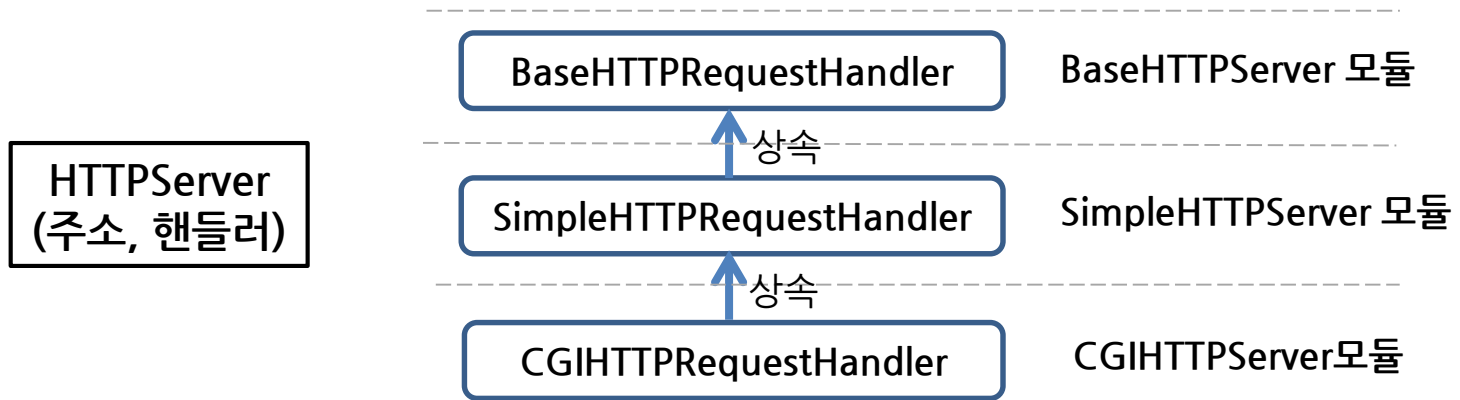
- BaseHTTPRequestHandler 또는 SimpleHTTPRequestHandler 클래스

→ HTTPServer 클래스와 핸들러 클래스를 사용하여 직접 서버 동작을 사용자 정의할 수 있음

10주차 정리. http.server 모듈 개요

■ http.server 모듈

- 파이썬은 HTTP 서버를 구현하기 위한 기본 모듈을 제공 → http.server
- 간단한 HTTP 서버(웹 서버)를 구현하고자 사용하는 모듈



- http.server가 제공하는 핸들러의 종류(어느 정도 구현이 되어 있는가의 차이)

종류	핸들러 클래스	특징
가장 기본 기본적인 프레임만 제공, 사용자가 로직 직접 추가	BaseHTTPRequestHandler	- 어떤 명령 메소드도 구현되지 않은 기본 서버 *메소드 오버라이딩(Method Overriding) 사용 :부모 클래스의 메소드를 상속받아 자식 클래스에서 재정의하는 방식
정적 파일 서버 GET은 구현되어 있음	SimpleHTTPRequestHandler	- BaseHTTPServer의 서브클래스 - GET, HEAD method 구현된 서버
스크립트 실행용 GET, POST 구현	CGIHTTPRequestHandler	- SimpleHTTPServer의 서브클래스 - POST method구현

■ HTTP 요청 메소드

메소드	특징
GET	- 리소스를 얻어 오기 위한 요청 - 바디 사용 없이 쿼리 스트링으로 요청 데이터 전달 예) <code>https://localhost:8089?height=170&weight=60</code>
POST	- 새로운 리소스를 생성하기 위한 요청 - 바디를 사용해 요청 데이터 전달 (폼데이터) 예) <code>height=170&weight=60</code>
PUT	- 리소스 수정 요청(원래 자원에서 새로운 정보 추가, 삭제하기 위한 요청) - 바디를 사용해 데이터 전달
DELETE	- 리소스를 삭제하기 위한 요청 - 바디 사용 없이 쿼리 스트링으로 데이터 전달

10주차 정리. http.server 모듈을 이용한 간단한 HTTP 서버 프로그래밍

■ 실습1) http.server 모듈의 기본 사용법 (파일명: httpServer.py)

```
1 # http.server 모듈을 이용한 간단한 웹 서버 구현
2 from http.server import HTTPServer, BaseHTTPRequestHandler
3
4 # 웹 페이지 작성(index.html)
5 html_index = '''
6     <html>
7     <body>
8         <h1>This is simple my HTTP Server using http.server</h1>
9     </body>
10 </html>
11 '''
12
13 # base 핸들러 상속받아 자체 핸들러 구현: 클라이언트 요청에 대한 처리 담당
14 class myHandler(BaseHTTPRequestHandler):
15     # 서버로 들어오는 모든 URL의 GET 요청에 대해 동일한 웹 페이지 전송
16     def do_GET(self): # 빈 로직을 상속받아 메소드 오버라이딩으로 자체 구현
17         print("GET 요청들어옴")
18         self.send_response(200) # status line(HTTP/1.1 200 OK)
19         self.send_header("Content-type", "text/html; charset=utf-8") # 응답바디의 타입지정
20         self.end_headers() # 더 이상 헤더없음. blank line(\r\n) 추가
21         # 클라이언트로 전송할 응답 바디 전송
22         self.wfile.write(html_index.encode('utf-8'))
23
24 if __name__ == "__main__":
25     address = ('localhost', 8080)
26     # HTTP 서버 생성: HTTPServer(주소, 핸들러)
27     server = HTTPServer(address, myHandler)
28     print(f"웹 서버가 {address}로 서비스 되고 있습니다")
29     server.serve_forever() # 서버시작(looping)
```

This is simple my HTTP Server using http.server

10주차 정리. http.server 모듈을 이용한 BMI 측정 웹 서버 구현

■ 응용실습) HTTP 서버 모듈의 GET,POST를 이용한 BMI 측정 웹 서버 구현

GET방식- URL 쿼리스트링으로 전송

http://localhost:8089/get/bmi?height=170&weight=60

홈 페이지 (/index.html)
http://localhost:8089/

BMI(체질량지수) Calculator Welcome Page

GET 방식으로 파라미터 전달

POST 방식으로 파라미터 전달

GET 방식으로 파라미터(키, 몸무게) 전송하기

170

60

전송하기

POST방식- 바디 폼데이터 전송

http://localhost:8089/post/bmi

POST 방식으로 파라미터(키, 몸무게) 전송하기

170

60

전송하기

height=170&weight=60

응답 페이지(html)

당신의 체질량지수는 20.76 입니다

10주차 정리. http.server 모듈을 이용한 BMI 측정 웹 서버 구현

- 응용실습) HTTP 서버 모듈의 GET,POST를 이용한 BMI 측정 웹 서버 구현

파일명: httpServer_bmi_이름.py

```
1 # http.server 모듈을 이용한 BMI 측정 웹 서버 구현(GET, POST방식처리)
2 from http.server import HTTPServer, BaseHTTPRequestHandler
3
4 # URL 구성요소별 파싱하는 모듈(URL:프로토콜, 도메인, 파라미터(쿼리))
5 from urllib import parse
6
7 # 웹 서버 를 페이지 제작(첫 화면)
8 html_index = '''
9     <!doctype html>
10    <html>
11        <body>
12            <h1>BMI(체질량지수) Calculator Welcome Page</h1>
13            <h3><a href="/get">GET 방식으로 파라미터 전달</a></h3>
14            <h3><a href="/post">POST 방식으로 파라미터 전달</a></h3>
15        </body>
16    </html>
17 '''
```

BMI(체질량지수) Calculator Welcome Page

[GET 방식으로 파라미터 전달](#)

[POST 방식으로 파라미터 전달](#)

10주차 정리. http.server 모듈을 이용한 BMI 측정 웹 서버 구현

■ 응용실습) HTTP 서버 모듈의 GET,POST를 이용한 BMI 측정 웹 서버 구현

```
18 # GET 요청으로 파라미터를 전송하는 웹 페이지 제작
19 html_get = '''
20     <!doctype html>
21     <html>
22         <body>
23             <h1>GET 방식으로 파라미터(키, 몸무게) 전송하기</h1>
24             <form method="GET" action="/get/bmi">
25                 <p><input type="text" name="height" placeholder="키(Cm)를 입력하세요"></p>
26                 <p><input type="text" name="weight" placeholder="몸무게(Kg)을 입력하세요"></p>
27                 <p><input type="submit" value="전송하기"></p>
28             </form>
29         </body>
30     </html>
31 '''
32 # POST 요청으로 파라미터를 전송하는 웹 페이지 제작
33 html_post = '''
34     <!doctype html>
35     <html>
36         <body>
37             <h1>POST 방식으로 파라미터(키, 몸무게) 전송하기</h1>
38             <form method="POST" action="/post/bmi">
39                 <p><input type="text" name="height" placeholder="키(Cm)를 입력하세요"></p>
40                 <p><input type="text" name="weight" placeholder="몸무게(Kg)을 입력하세요"></p>
41                 <p><input type="submit" value="전송하기"></p>
42             </form>
43         </body>
44     </html>
45 '''
46
```

GET 방식으로 파라미터(키, 몸무게) 전송하기

POST 방식으로 파라미터(키, 몸무게) 전송하기

10주차 정리. http.server 모듈을 이용한 BMI 측정 웹 서버 구현

■ 응용실습) HTTP 서버 모듈의 GET,POST를 이용한 BMI 측정 웹 서버 구현

```
47 # base 핸들러 상속받아 자체 핸들러 제작
48 class myHandler(BaseHTTPRequestHandler):
49     def do_GET(self): # 빈 로직 상속받아 메소드 오버라이딩으로 자체 구현
50         print("GET 요청 들어옴")
51         # 경로의 끝이 루트('/') 인 경우 (기본요청)
52         if self.path.endswith('/'):
53             self.send_response(200) # status line: HTTP/1.1 200 OK
54             self.send_header("Content-type", "text/html; charset=utf-8") #바디 type 지정
55             self.end_headers() # 더이상 헤더없음. blank line 추가
56             # 응답메시지의 바디 전송
57             self.wfile.write(html_index.encode('utf-8'))
58
59         # 경로의 끝이 ('/get') 인 경우
60         if self.path.endswith('/get'):
61             self.send_response(200) # status line: HTTP/1.1 200 OK
62             self.send_header("Content-type", "text/html; charset=utf-8") #바디 type 지정
63             self.end_headers() # 더이상 헤더없음. blank line 추가
64             # 응답메시지의 바디 전송
65             self.wfile.write(html_get.encode('utf-8'))
66
67         # 경로의 끝이 ('/post') 인 경우
68         if self.path.endswith('/post'):
69             self.send_response(200) # status line: HTTP/1.1 200 OK
70             self.send_header("Content-type", "text/html; charset=utf-8") #바디 type 지정
71             self.end_headers() # 더이상 헤더없음. blank line 추가
72             # 응답메시지의 바디 전송
73             self.wfile.write(html_post.encode('utf-8'))
74
```

10주차 정리. http.server 모듈을 이용한 BMI 측정 웹 서버 구현

■ 응용실습) HTTP 서버 모듈의 GET,POST를 이용한 BMI 측정 웹 서버 구현

```
75     # 경로의 시작이 ('/get/bmi') 인 경우 (경로의 끝에 쿼리스트링 추가되어 있음)
76     if self.path.startswith('/get/bmi'):
77         # URL(/get/id?key=value)에서 쿼리 스트링 파싱(분리)
78         url = parse.urlparse(self.path) # URL을 구성요소별 분리
79         q_string = url.query # 쿼리스트링의 폼 데이터 저장(key=value)
80         # parse_qs() : 폼데이터는 기본적인 문자열 --> 딕셔너리로 변환
81         params= parse.parse_qs(q_string)
82         print(f"파라미터 딕셔너리변환: {params}")
83         height = float(params.get('height',[''])[0]) # height 첫번째 값 추출
84         weight = float(params.get('weight',[''])[0]) # weight 첫번째 값 추출
85         print(f"카: {height}, 몸무게: {weight}")
86         bmi = round(weight/(height/100)**2, 2) #소수점 2자리까지만 출력
87
88         # 추출한 name값을 포함해서 응답 페이지 보내기
89         self.send_response(200) # status line: HTTP/1.1 200 OK
90         self.send_header("Content-type", "text/html; charset=utf-8") #바디 type지정
91         self.end_headers() # 더이상 헤더없음. blank line 추가
92         # 응답메시지의 바디 전송
93         response_body = f"<h2>당신의 체질량지수는 {bmi} 입니다</h2>"
94         self.wfile.write(response_body.encode('utf-8'))
95
```

10주차 정리. http.server 모듈을 이용한 BMI 측정 웹 서버 구현

■ 응용실습) HTTP 서버 모듈의 GET,POST를 이용한 BMI 측정 웹 서버 구현

```
96 def do_POST(self): # 빈 로직 상속받아 메소드 오버라이딩으로 자체 구현
97     print("POST 요청 들어옴")
98     # 경로의 끝이 루트('/post/bmi') 인 경우
99     if self.path.endswith('/post/bmi'):
100         # 웹브라우저 요청메시지 바디의 폼데이터(파라미터) 저장
101         content_length = int(self.headers['Content-length']) # 바디 길이지정
102         data = self.rfile.read(content_length).decode('utf-8') # 바디 폼데이터 저장
103         if data is not None:
104             # parse_qs() : 폼데이터는 기본적인 문자열 --> 딕셔너리로 변환
105             dic_data = parse.parse_qs(data)
106             print(f"폼데이터 딕셔너리변환: {dic_data}")
107             height = float(dic_data.get('height',[''])[0]) # height의 첫번째 값 추출
108             weight = float(dic_data.get('weight',[''])[0]) # height의 첫번째 값 추출
109             print(f"카: {height}, 몸무게: {weight}")
110             bmi = round(weight/(height/100)**2, 2) #소수점 2자리까지만 출력
111
112         # 바디에서 추출한 name을 포함해서 응답 페이지 보내기
113         self.send_response(200) # status line: HTTP/1.1 200 OK
114         self.send_header("Content-type", "text/html; charset=utf-8") #바디 type지정
115         self.end_headers() # 더이상 헤더없음. blank line 추가
116         # 응답메시지의 바디 전송
117         response_body = f"<h2>당신의 체질량지수는 {bmi} 입니다</h2>"
118         self.wfile.write(response_body.encode('utf-8'))
119
```

10주차 정리. http.server 모듈을 이용한 BMI 측정 웹 서버 구현

- 응용실습) HTTP 서버 모듈의 GET,POST를 이용한 BMI 측정 웹 서버 구현

```
120 if __name__ == "__main__":  
121     address = ('localhost', 8089)  
122     # HTTP 서버 생성 , HTTPServer(주소, 핸들러)  
123     server = HTTPServer(address, myHandler)  
124     print(f" BMI 측정 웹 서버가 {address}로 서비스 되고 있습니다")  
125     server.serve_forever()    # 서버 시작(looping)
```

1. 파이썬 requests 모듈 기본 사용법

■ Python requests 모듈 사용하기

- requests 모듈은 Python용 HTTP 라이브러리
 - HTTP 요청(Request)을 보내고 응답(Response)을 받는데 사용되는 라이브러리
 - 특정 웹사이트에 접속하여 페이지의 HTML 소스코드를 받아서 필요한 정보를 추출할 때 사용



- requests 모듈이 없다면 설치 필요
 - 주피터노트북 터미널 창 혹은 윈도우 cmd 창에서 모듈 다운로드 받기

pip install requests

1. 파이썬 requests 모듈 기본 사용법

■ Python requests 모듈 사용하기

- requests 모듈 제공 API
 - 각 요청방식(Method)에 맞는 함수 제공

GET 방식 - `requests.get()`

POST 방식 - `requests.post()`

PUT 방식 - `requests.put()`

DELETE 방식 - `requests.delete()`

```
# requests 모듈 사용하기  
  
import requests  
  
requests.get('https://google.com')
```

`<Response [200]>`

← 상태코드를 응답으로 받음

1. 파이썬 requests 모듈 기본 사용법

■ Python requests 모듈 사용하기

- <https://httpbin.org> 를 이용해서 requests 모듈 사용법을 익혀봅시다
 - * *httpbin.org* 는 *http* 메서드 테스트 사이트 (보낸 요청을 그대로 돌려줌)



HTTP request

HTTP response

httpbin.org 0.9.2
[Base URL: httpbin.org/]
A simple HTTP Request & Response Service.
Run locally: `$ docker run -p 80:80 kennethreitz/httpbin`
[the developer - Website](#)
[Send email to the developer](#)

Schemes

HTTPS ▼

HTTP Methods Testing different HTTP verbs

DELETE /delete The request's DELETE parameters.

GET /get The request's query parameters.

PATCH /patch The request's PATCH parameters.

POST /post The request's POST parameters.

PUT /put The request's PUT parameters.

1. 파이썬 requests 모듈 기본 사용법

■ Python requests 모듈 사용하기 - GET 방식

1) GET 요청 보내고 응답 받기 : requests.get(url) 요청하기

```
# Python requests 모듈 사용하기
# HTTP 요청(Request)을 보내고 응답(Response)을 받기위한 라이브러리
import requests, json
```

```
# GET 요청보내고 응답받기
# requests.get(url)
url = 'https://httpbin.org/get'
```

```
response = requests.get(url)
# 응답메시지 출력(JSON type)
print(response.text)
# 응답코드만 출력
print(response.status_code)
# 응답헤더만 출력
print(response.headers)
# 헤더중 Content-Type만 출력
print(response.headers['Content-Type'])
```

```
{
  "args": {},
  "headers": {
    "Accept": "*/*",
    "Accept-Encoding": "gzip, deflate, br, zstd",
    "Host": "httpbin.org",
    "User-Agent": "python-requests/2.32.3",
    "X-Amzn-Trace-Id": "Root=1-6a068622-0951a8f35d4ba46b0a14ea85"
  },
  "origin": "175.212.188.154",
  "url": "https://httpbin.org/get"
}
```

200

```
{'Date': 'Fri, 15 May 2026 02:34:10 GMT', 'Content-Type': 'application/json', 'Content-Length': '319', 'Connection': 'keep-alive', 'Server': 'gunicorn/19.9.0', 'Access-Control-Allow-Origin': '*', 'Access-Control-Allow-Credentials': 'true'}
```

application/json

1. 파이썬 requests 모듈 기본 사용법

- Python requests 모듈 사용하기 - GET 방식

2) GET요청으로 받은 응답메시지 바디에서 원하는 정보만 처리

- 받은 JSON 파일(혹은 다른 형식의 파일)을 딕셔너리로 변환 후 원하는 정보 추출

```
# 응답메시지 JSON -> 딕셔너리로 변환
dic_data = response.json()
print(dic_data)
# 딕셔너리에서 'url'만 출력하기
print(dic_data['url'])
```

```
{'args': {}, 'headers': {'Accept': '*/*', 'Accept-Encoding': 'gzip, deflate, br, zstd', 'Host': 'httpbin.org', 'User-Agent': 'python-requests/2.32.3', 'X-Amzn-Trace-Id': 'Root=1-6a0692cb-2fbb944f55304efa695baf98'}, 'origin': '175.212.188.154', 'url': 'https://httpbin.org/get'}
https://httpbin.org/get
```

1. 파이썬 requests 모듈 기본 사용법

■ Python requests 모듈 사용하기 - GET 방식

3) GET 요청에 파라미터 추가하기 : requests.get(url, params)

- 파라미터는 URL의 쿼리 스트링으로 전송 (url + ?query 스트링)

```
# GET 요청시 파라미터 추가하기
# requests.get(url, params)

import requests, json

url = 'https://httpbin.org/get'
# 파라미터 정의(key:value)
params = {"name": "Hello", "data": "World"}

# GET 요청시 파라미터 추가- requests.get(url, params)
response = requests.get(url, params=params)
dic_data = response.json() # JSON -> Dic
print("GET요청 URL:", dic_data['url'])
```

GET요청 URL: <https://httpbin.org/get?name=Hello&data=World>

query 스트링

1. 파이썬 requests 모듈 기본 사용법

■ Python requests 모듈 사용하기 - POST 방식

4) POST 요청하고 응답받기

- 파라미터는 바디에 폼 데이터로 전송

```
# POST 요청시 파라미터 추가하기
# requests.post(url, data)

import requests, json

url = 'https://httpbin.org/post'
# 요청메시지의 바디 정의(일반 폼 데이터 형식)
body_data = {"name": "Hello", "data": "World"}

# POST 요청으로 바디를 보내고 응답받기
# requests.post(url, body_data)
response = requests.post(url, data=body_data)
dic_data = response.json() # JSON -> Dic
print("POST 요청 URL:", dic_data['url'])
print("POST 요청 바디:", dic_data['form'])
```

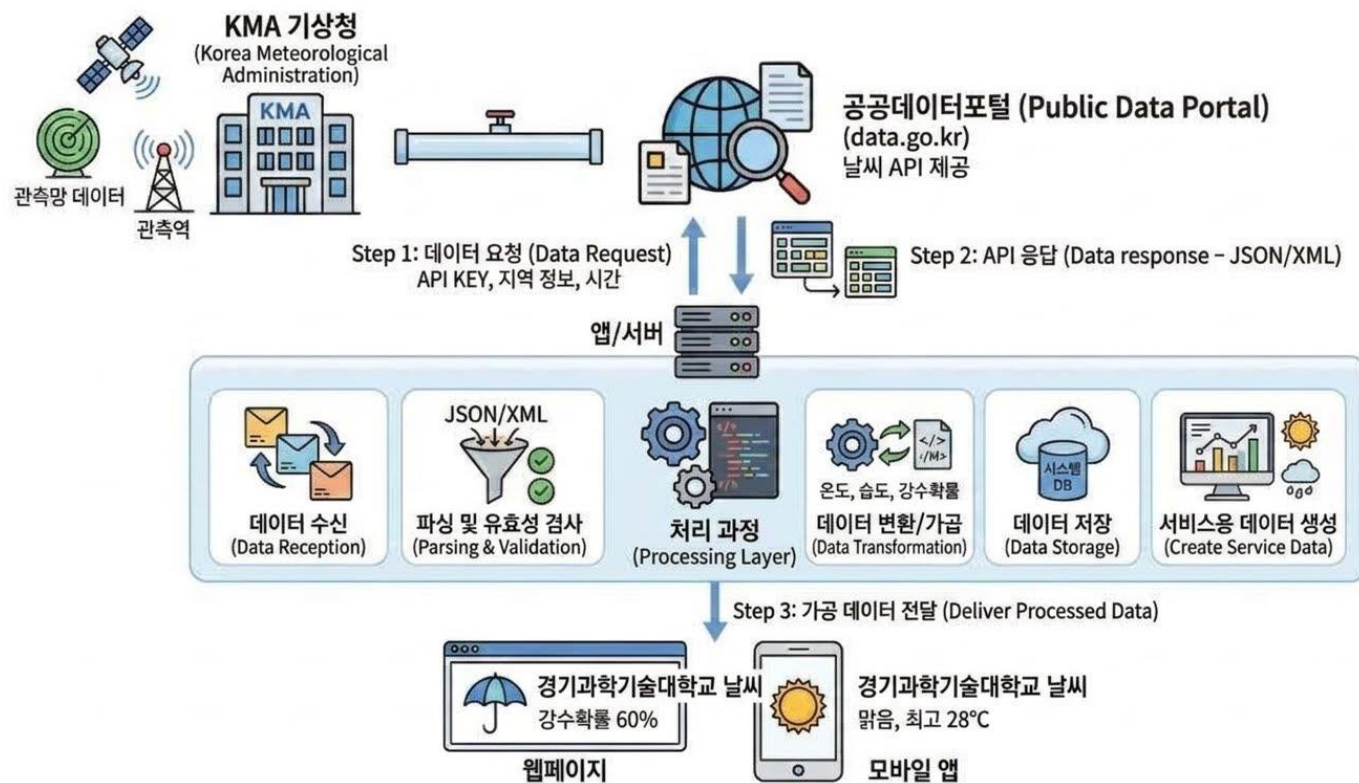
POST 요청 URL: <https://httpbin.org/post>

POST 요청 바디: {'data': 'World', 'name': 'Hello'}

2. 공공데이터포털 기상청 API 이용하기

■ 기상청 API로 우리지역 날씨 정보 알아내기

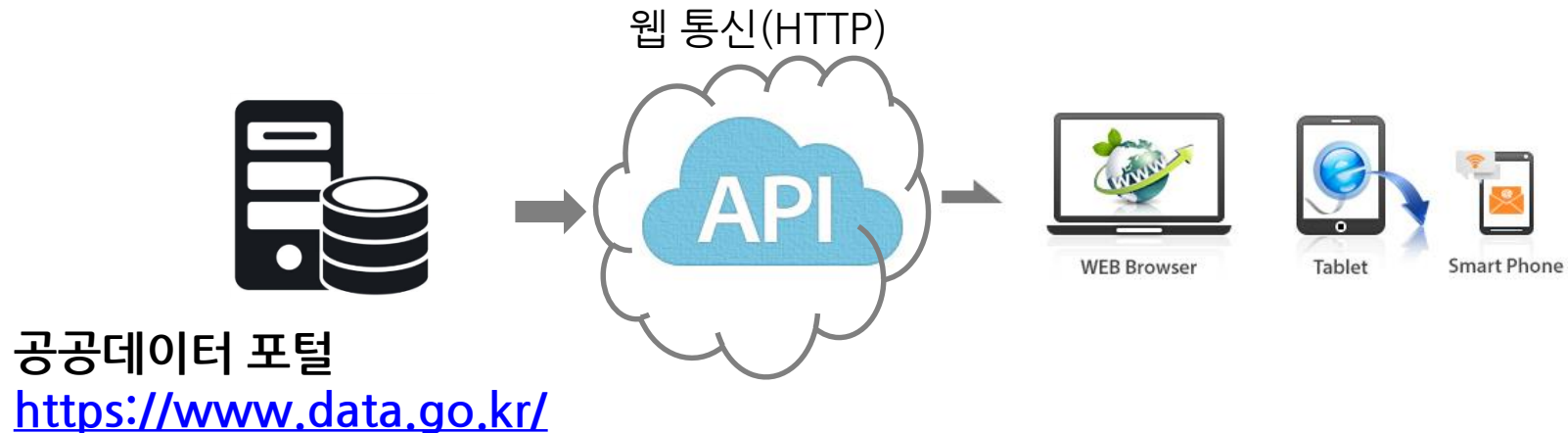
- HTTP 통신을 통해 기상청 서버로부터 날씨 정보를 받아서 서비스에 이용해 봅시다



2. 공공데이터포털 기상청 API 이용하기

■ 공공데이터와 OpenAPI

- 공공데이터란 공공기관이 만들어내는 모든 자료나 정보
- 공공데이터는 “공공데이터포털”을 통해서 파일 다운로드나 **Open API** 형태로 활용
 - * OpenAPI(Open Application Programming Interface, 공개 API)란 누구나 사용할 수 있도록 공개된 API
 - * **API**란 두 애플리케이션 간 요청과 응답을 사용하여 서로 통신하는 방법(형식)을 정의



2. 공공데이터포털 기상청 API 이용하기

■ 공공데이터와 OpenAPI

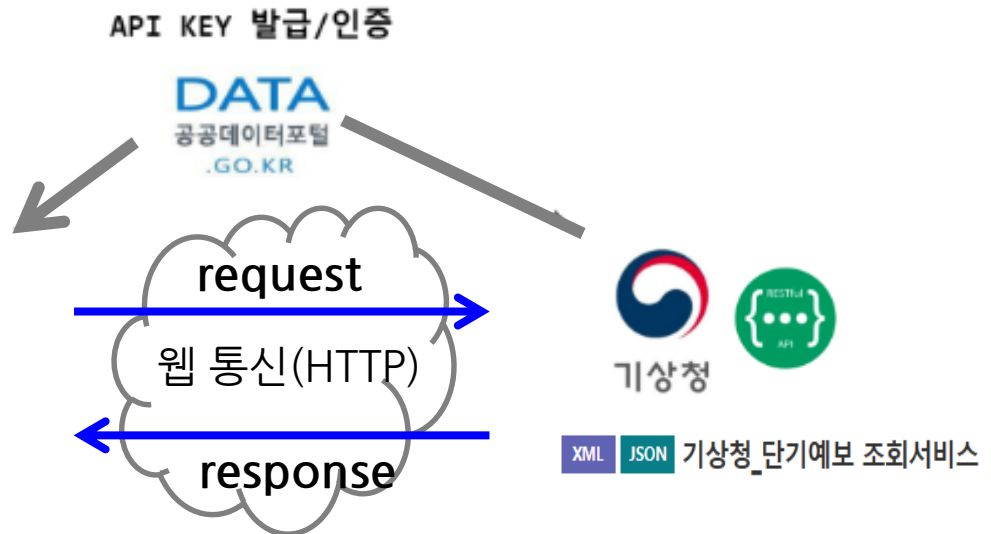
- HTTP 통신(웹 통신)을 통해 기상청 서버로 부터 기상정보를 받아서 처리해 보시다
- 공공데이터포털 이용시 통신에 필요한 **API Key 발급**이 필요합니다

Python code

```
import requests

url = 'http://apis.data.go.kr/1360000/VilageFcstInfoSe
params = {'serviceKey' : '서비스키', 'pageNo' : '1', 'nur

response = requests.get(url, params=params)
print(response.content)
```



3. 공공데이터 포털 가입하기 및 인증키 받기

■ 공공데이터 포털 회원가입

‘일반회원’ 선택 후 이름 및 이메일 입력 후 [가입확인] 버튼 클릭

 이 누리집은 대한민국 공식 전자정부 누리집입니다.

[로그인](#) [회원가입](#) [사이트맵](#) [ENGLISH](#)

DATA 공공데이터포털 . GO . KR

[데이터찾기](#) [국가데이터맵](#) [데이터요청](#) [데이터활용](#) [정보공유](#) [이용안내](#)

회원가입을 하시면 공공데이터포털을 통해 국가가 보유하고 있는 유용한 공공데이터와 오픈API 등 모든 정보를 자유롭게 이용하실 수 있습니다.

STEP1 가입확인 및 회원구분

STEP2 정보입력 및 약관동의

1. 회원가입여부 확인을 위해 이름, 이메일 주소를 입력해주세요.

이름

※ 실명으로 가입하지 않을 경우 공공데이터 제공신청, 분쟁조정신청 등 일부 서비스에 대해 불이익이 있을 수 있습니다.

이메일

※ 이메일 주소 입력은 영문키 상태에서 기재해주세요.

가입확인

3. 공공데이터 포털 가입하기 및 인증키 받기

■ 일반회원으로 회원가입 진행

STEP1 가입확인 및 회원구분		STEP2 정보입력 및 약관동의	
기본정보 입력 *표시는 필수 입력항목입니다.			
* 아이디		중복확인	
4~20자 이내 영문 소문자, 숫자만 가능합니다.			
* 비밀번호			
반복, 연속문자, 개인정보 관련은 사용불가 숫자, 영문자, 특수문자(!, @, #, \$, %, ^, &, +, =, ~, *) 조합 9자이상 ~ 20자 이하			
* 비밀번호 확인			
℥인정보입력 입력한 이메일로 들어가서 인증번호 확인			
* 이름			
* 이메일		이메일 인증	☐ 이메일 수신 동의
수신거부 시 일부 서비스에 제한이 있을 수 있습니다. 비밀번호 초기화 메일 수신 등에 반드시 필요한 정보이므로 정확히 이메일 주소를 입력해주세요.			

3. 공공데이터 포털 가입하기 및 인증키 받기

■ 이메일 인증 및 약관 동의

*이메일 인증번호

인증번호 확인

전화번호

*휴대전화번호

☐ SMS 수신 동의

수신거부 시 일부 서비스에 제한이 있을 수 있습니다.

*일반회원 인증

휴대폰 인증

본인 인증하기

아이핀 인증

본인 인증하기

*자동등록방지

cdx6a

보안문자를 입력하십시오

(필수) 개인정보 수집 및 이용 ☒ 동의 ☐ 동의안함

공공데이터포털에서 현황조사 등에 수집된 정보가 활용될 수 있습니다.

- . 정보 수집 목적
- 공공데이터포털 및 데이터활용 관련 정보 제공
 - 공공데이터 활용현황조사

※ 선택 목적에 동의하지 않는 경우, 포털 및 데이터 활용 관련 정보 제공과 활용 현황조사 대상에서 제외됩니다.

(선택) 공공데이터 활용현황조사 등에 수집된 정보 활용 ☒ 동의 ☐ 동의안함

☐ 전체 약관에 동의합니다.

취소

작성 완료

확인버튼 2번을 클릭하고 나면 가입이 완료됩니다.

회원 가입이 완료되었습니다.

공공데이터포털에서 제공하는 모든 정보를 자유롭게 이용할 수 있습니다.

3. 공공데이터 포털 가입하기 및 인증키 받기

■ 로그인 접속 후 '기상청 단기예보' 검색

The image shows two screenshots from the Korea Public Data Portal (공공데이터포털). The top screenshot is the login page, titled '공공데이터포털 로그인' (Public Data Portal Login). It features a blue header with navigation links: '로그인' (Login), '회원가입' (Sign Up), '사이트맵' (Site Map), and 'ENGLISH'. Below the header, there are links for '데이터찾기' (Find Data), '국가데이터맵' (National Data Map), '데이터요청' (Data Request), '데이터활용' (Data Use), '정보공유' (Information Sharing), and '이용안내' (Usage Guide). The main content area has a white box with the title '공공데이터포털 로그인' and the subtitle '국민과 함께 하는 공공데이터포털에 오신 것을 환영합니다' (Welcome to the Public Data Portal, where we serve the people together). Below this, there are two input fields: '아이디' (ID) and '비밀번호' (Password), both highlighted with a red border. There is also a checkbox for '아이디 저장' (Save ID) and a link for '아이디·비밀번호 찾기' (Find ID/Password). The bottom screenshot shows the search results page. A pink arrow points from the login page to the search page. The search bar contains the text '기상청 단기예보' (Weather Service Short-term Forecast), which is highlighted with a red border. Below the search bar, there is a dropdown menu showing '기상청' (Weather Service). To the right of the search bar, there is a magnifying glass icon and a button labeled '인기검색어' (Popular Search Terms). Below this, there is a list of popular search terms, including '4. 미세먼지' (4. PM2.5), which is highlighted with a blue background. At the bottom right, there is a link for '① 검색도움말' (1. Search Help).

공공데이터포털 로그인

국민과 함께 하는 공공데이터포털에 오신 것을 환영합니다

아이디

비밀번호

☐ 아이디 저장

아이디·비밀번호 찾기

기상청 단기예보

인기검색어

4. 미세먼지

① 검색도움말

3. 공공데이터 포털 가입하기 및 인증키 받기

■ '오픈 API' 선택 후 '기상청_단기예보' 활용신청하기

"기상청 단기예보"에 대해 총 99건이 검색되었습니다.

조건검색 [초기화](#)

분류체계	서비스유형	제공기관유형	확장자	
국가중점데이터 분류 조건 추가하기 +				
전체(99건)	파일데이터(66건)	오픈 API(33건)	표준데이터셋0개(0건)	연계데이터(0건)

오픈 API (33건)

과학기술 국가행정기관 [미리보기](#)

XML JSON 기상청_단기예보 조회서비스

초단기실황, 초단기예보, 단기((구)동네)예보, 예보버전 정보를 조회하는 서비스입니다.

제공기관 기상청 수정일 2026-03-26 조회수 603641 활용신청 56031 키워드 단기예보,초단기실황,초단기예보,동네예보,읍면동

활용신청

과학기술 국가행정기관 국가중점 [미리보기](#)

XML JSON 기상청_단기예보 통보문 조회서비스

발표관서나 예보구역 정보를 조건으로 기상개황, 육상예보, 해상예보를 조회하는 서비스 (참고) 일반적인 단기예보란 예보기간과 구역을 시·공간적으로 세분화하여 발표하는 예보입니...

제공기관 기상청 수정일 2026-03-26 조회수 74660 활용신청 3587 키워드 기상개황,육상예보,해상예보,기온,해상상태

활용신청

3. 공공데이터 포털 가입하기 및 인증키 받기

■ 활용신청 하기

활용목적 입력하고 이용허락범위 동의 후 활용신청하기

활용목적 선택

*활용목적

☒ 웹 사이트 개발 ☐ 앱개발 (모바일,솔루션등) ☐ 기타 ☐ 참고자료 ☐ 연구(논문 등)

데이터 분석 활용

0/250

첨부파일

파일 선택

Drag & Drop으로 파일을 선택 가능합니다.

상세기능정보 선택

☒	상세기능	설명
☒	국토교통부_아파트 매매 자료	지역코드와 기간을 이용하여 해당기간, 해당지역의 아파트 매매 신고자료를 제공하는 아파트 매매 신고 정보 조회

라이선스 표시

*이용허락범위

이용허락범위 제한 없음

☒ 동의합니다.

취소

활용신청

www.data.go.kr 내용:

신청이 완료되었습니다. 신청된 API는 1~2시간 후 호출 가능합니다.

※ 일부 API에서 24시간 후 인증이 가능한 API가 있습니다.
빠른 시일내에 모든 API가 1시간 이내 인증 가능하도록 개선하겠습니다

확인

3. 공공데이터 포털 가입하기 및 인증키 받기

■ 마이페이지에서 인증키 발급 확인

이 누리집은 대한민국 공식 전자정부 누리집입니다.

로그아웃 마이페이지 사이트맵 ENGLISH

DATA 공공데이터포털 .GO.KR 데이터찾기 국가데이터맵 데이터요청 데이터활용 정보공유 이용안내

김인철님, 반갑습니다.
[회원정보 수정 >](#)

파일데이터 0건

API신청 2건

관심데이터 0건

전체 2건 >	신청 0건 >	활용 2건 >	만료·중지 0건 >
전체 API활용 신청 현황	신청중인 단계	승인되어 활용중인 단계	활용기간이 만료·중지된 단계
	보류 0건 반려 0건	변경신청 0건	만료 0건 중지 0건
			※만료된 경우 연장 신청 필요

총 2건

과학기술 기상청

연장신청 [승인] 기상청_단기예보 조회서비스

계정 개발 신청일 2026-05-08 만료예정일 2028-05-08

3. 공공데이터 포털 가입하기 및 인증키 받기

■ 인증키 확인

Decoding: 인간이 읽을 수 있는 원래 상태의 키
Encoding: URL 전용 아스키 표준 문자로 변환(%XX 형태) 완료한 상태의 키
→ Python requests 모듈은 전송 직전 자체적으로 디코딩 키를 인코딩 변환하기에
Decoding 인증키 사용

개발계정 상세보기

기본정보

데이터명	기상청_단기예보 조회서비스 상세설명		
서비스유형	REST	심의여부	자동승인
신청유형	개발계정 연장신청	처리상태	승인
활용기간	2024-05-08 ~ 2028-05-08		

서비스정보

참고문서	기상청41 단기예보 조회서비스 오픈API활용가이드 2510.zip
데이터포맷	JSON+XML
End Point	https://apis.data.go.kr/1360000/VilageFcstInfoService_2.0

API 환경 또는 API 호출 조건에 따라 인증키가 적용되는 방식이 다를 수 있습니다.
포털에서 제공되는 **Encoding/Decoding** 된 인증키를 적용하면서 구동되는 키를 사용하시기 바랍니다.
* 향후 포털에서 더 명확한 정보를 제공하기 위해 노력하겠습니다.

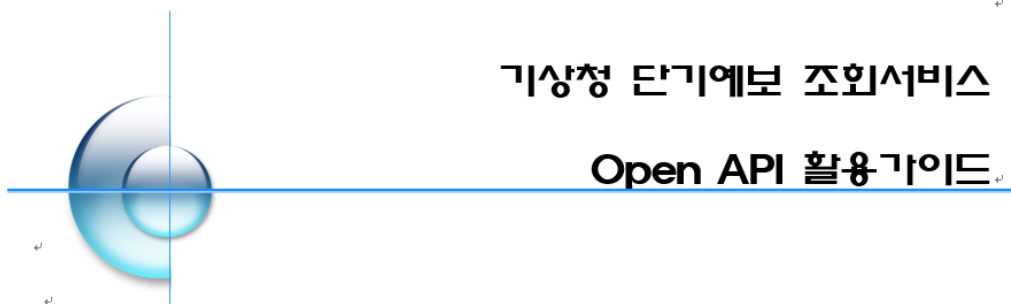
일반 인증키 (Encoding)	RW3Fc8oL6uMihndTdk7Aanj1Zql%2BPX%2FfePoPbLJx2DeUp%2F7CLP9dUvFfgYasMDUbpzWUseHQvZK9WRmc82GSuQ%3D%3D
일반 인증키 (Decoding)	RW3Fc8oL6uMihndTdk7Aanj1Zql+PX/FePoPbLJx2DeUp/7CLP9dUvFfgYasMDUbpzWUseHQvZK9WRmc82GSuQ==

Decoding 인증키 사용

4. 공공데이터포털 기상청 API 이용하기

■ API 형식 확인하기

- 기상청41_단기예보 조회서비스_오픈API활용가이드.txt



- API(Application Programming Interface)를 사용하려면 API 문서를 확인해야 함
- API 문서는 API가 제공하는 기능, 사용 방법, 요청 및 응답 형식 등 필요한 모든 정보를 담고 있음
- API 문서를 통해 어떤 API를 사용해야 하는지, 어떻게 사용해야 하는지, 어떤 데이터를 요청하고 응답으로 받는지 등을 정확하게 파악할 수 있음

4. 공공데이터포털 기상청 API 이용하기

■ 기상청 단기예보 코딩 가이드 살펴보기

[초단기실황조회] 상세기능명세

실습에 사용할 서비스

[초단기실황]은 현재의 날씨 상태를
1시간 간격으로 제공하는 실제 관측값
범위: 최근 1일(24시간) 이내

번호	API 명(국문)	상세기능명(영문)	상세기능명(국문)
1	단기예보 조회서비스	getUltraSrtNcst	초단기실황조회
2		getUltraSrtFcst	초단기예보조회
3		getVilageFcst	단기예보조회
4		getFcstVersion	예보버전조회

1) [초단기실황조회] 상세기능명세

a) 상세기능정보

상세기능 번호	1	상세기능 유형	조회 (목록)
상세기능명(국문)	초단기실황조회		
상세기능 설명	실황정보를 조회하기 위해 발표일자, 발표시각, 예보지점 X 좌표, 예보지점 Y 좌표의 조회 조건으로 자료구분코드, 실황값, 발표일자, 발표시각, 예보지점 X 좌표, 예보지점 Y 좌표의 정보를 조회하는 기능		
Call Back URL	http://apis.data.go.kr/1360000/VilageFcstInfoService_2.0/getUltraSrtNcst		
최대 메시지 사이즈	[1764] byte		
평균 응답 시간	[100] ms	초당 최대 트래잭션	[30] tps

사용할 기본 URL 확인

4. 공공데이터포털 기상청 API 이용하기

■ 기상청 단기예보 코딩 가이드 살펴보기

- 쿼리스트링의 파라미터 형식 (params)

b) 요청 메시지 명세

URL = 기본URL?쿼리스트링 파라미터

항목명(영문)	항목명(국문)	항목크기	항목구분	샘플데이터	항목설명
serviceKey	인증키	100	1	인증키 (URL Encode)	공공데이터포털에서 발급받은 인증키
numOfRows	한 페이지 결과 수	4	1	10	한 페이지 결과 수 Default: 10
pageNo	페이지 번호	4	1	1	페이지 번호 Default: 1
dataType	응답자료형식	4	0	XML	요청자료형식(XML/JSON) Default: XML
base_date	발표일자	8	1	20210628	'21년 6월 28일 발표
base_time	발표시각	4	1	0600	06시 발표(정시단위) -매시각 10분 이후 호출
nx	예보지점 X 좌표	2	1	55	예보지점의 X 좌표값 *별첨 엑셀 자료 참조
ny	예보지점 Y 좌표	2	1	127	예보지점의 Y 좌표값 *별첨 엑셀 자료 참조

← 서비스 키

← JSON 사용

기상실황 일시
(40분 이후 현재시간,
40분 이전 이전시간의
정시데이터)

기상실황 지점 격자좌표
(정왕동 지정)

- 정왕동 격자 위경도 좌표 (56, 122)

A	B	C	D	E	F	G	H	I	J	K	L	M
구분	행정구역코드	1단계	2단계	3단계	격자 X	격자 Y	경도(시)	경도(분)	경도(초)	위도(시)	위도(분)	위도(초)
kor	4139059100	경기도	시흥시	정왕1동	56	122	126	44	45.52	37	20	13.86

4. 공공데이터포털 기상청 API 이용하기

■ 초단기실황 서비스 발표시각

○ 초단기실황 발표시각 ↕

※ 매시간 정시에 생성되고 10분마다 최신 정보로 업데이트 ↕

기준 시간↕	생성시간↕	Base_time↕	API 제공 시간(~이후)↕	기준 시간↕	생성시간↕	Base_time↕	API 제공 시간(~이후)↕
00 시↕	00:00↕	0000↕	00:10↕	12 시↕	12:00↕	1200↕	12:10↕
01 시↕	01:00↕	0100↕	01:10↕	13 시↕	13:00↕	1300↕	13:10↕
02 시↕	02:00↕	0200↕	02:10↕	14 시↕	14:00↕	1400↕	14:10↕
03 시↕	03:00↕	0300↕	03:10↕	15 시↕	15:00↕	1500↕	15:10↕
04 시↕	04:00↕	0400↕	04:10↕	16 시↕	16:00↕	1600↕	16:10↕
05 시↕	05:00↕	0500↕	05:10↕	17 시↕	17:00↕	1700↕	17:10↕
06 시↕	06:00↕	0600↕	06:10↕	18 시↕	18:00↕	1800↕	18:10↕
07 시↕	07:00↕	0700↕	07:10↕	19 시↕	19:00↕	1900↕	19:10↕
08 시↕	08:00↕	0800↕	08:10↕	20 시↕	20:00↕	2000↕	20:10↕
09 시↕	09:00↕	0900↕	09:10↕	21 시↕	21:00↕	2100↕	21:10↕
10 시↕	10:00↕	1000↕	10:10↕	22 시↕	22:00↕	2200↕	22:10↕
11 시↕	11:00↕	1100↕	11:10↕	23 시↕	23:00↕	2300↕	23:10↕

- 초단기실황 데이터는 매시간 00분에 생성되어 매시간 10분에 최신정보 업데이트
- 매시간 10분 이후 API로 요청할 수 있으나 안전하게 매 시간 40분을 기점으로 데이터 요청
→ 예시) 12시 30분인 경우 40분 이전이므로 1100(1시간 이전 정시) 기상정보 요청

4. 공공데이터포털 기상청 API 이용하기

■ 기상청 단기예보 코딩 가이드 살펴보기

○ 응답메시지의 파라미터 형식

c) 응답 메시지 명세

항목명(영문)	항목명(국문)	항목크기	항목구분	샘플데이터	항목설명
numOfRows	한 페이지 결과 수	4	1	1	한 페이지당 표출 데이터 수
pageNo	페이지 번호	4	1	1	페이지 수
totalCount	데이터 총 개수	10	1	1	데이터 총 개수
resultCode	응답메시지 코드	2	1	00	응답 메시지코드
resultMsg	응답메시지 내용	100	1	NORMAL SERVICE	응답 메시지 설명
dataType	데이터 타입	4	1	XML	응답자료형식 (XML/JSON)
baseDate	발표일자	8	1	20210628	'21년 6월 28일 발표
baseTime	발표시각	6	1	0600	06시 발표(매 정시)
nx	예보지점 X 좌표	2	1	55	입력한 예보지점 X 좌표
ny	예보지점 Y 좌표	2	1	127	입력한 예보지점 Y 좌표
category	자료구분코드	3	1	RN1	자료구분코드 * 하단 코드값 정보 참조
obsrValue	실황 값	2	1	0	RN1, T1H, UUU, VVV, WSD 실수 또는 정수로 제공 * 하단 코드값 정보 참조

○ 응답메시지에서 추출할 데이터 (기온, 습도, 강수형태)

초단기실황	T1H	기온
	RN1	1시간 강수량
	UUU	동서바람성분
	VVV	남북바람성분
	REH	습도
	PTY	강수형태
	VEC	풍향
	WSD	풍속

- 강수형태(PTY) 코드 :
(초단기) 없음(0), 비(1), 비/눈(2), 눈(3),
빗방울(5), 빗방울눈날림(6), 눈날림(7)

4. 공공데이터포털 기상청 API 이용하기

■ 데이터 교환 형식: JSON 과 XML

- 데이터를 저장, 교환하기 위한 방법으로 계층적 구조, 다양한 프로그래밍언어에서 파싱할 수 있음

◦ JSON(JavaScript Object Notation)

- 가독성이 뛰어난 데이터 교환 형식
- 프로그래밍 언어나 플랫폼에 독립적 프로그래밍 언어에서 쉽게 이용
- 구조 간단하여 데이터 크기가 작음
- 키-값 쌍으로 이루어져 있음

```
{
  "name": "foktanma",
  "age": 30,
  "isStudent": false,
  "blog": ["hahahoho5915.tistory.com", "tistory.com"],
  "address": {
    "street": "123 Main St",
    "city": "Anytown",
    "postalCode": "12345"
  }
}
```

◦ XML(eXtensible Markup Language)

- 마크업 언어
- 스키마 지원으로 문서의 의미와 구조를 함께 저장할 수 있음
- 문서중심 데이터 교환에 유리

```
<Person>
  <Name>foktanma</Name>
  <Age>30</Age>
  <IsStudent>>false</IsStudent>
  <Blogs>
    <Blog>hahahoho5915.tistory.com</Blog>
    <Blog>tistory.com</Blog>
  </Blogs>
  <Address>
    <Street>123 Main St</Street>
    <City>Anytown</City>
    <PostalCode>12345</PostalCode>
  </Address>
</Person>
```

4. 공공데이터포털 기상청 API 이용하기

■ 포털에서 제공하는 샘플코드 확인

오픈API 상세

XML JSON 기상청_단기예보 조회서비스

샘플코드

Java

Javascript

C#

PHP

Curl

Objective-C

Python

Nodejs

R

Python3 샘플 코드

기상청_단기예보 조회서비스
페이지 제일 하단에서 참조

```
import requests
```

```
url = 'http://apis.data.go.kr/1360000/VilageFcstInfoService_2.0/getUltraSrtNcst'
```

```
params ={'serviceKey' : '서비스키', 'pageNo' : '1', 'numOfRows' : '1000', 'dataType' : 'XML', 'base_date' : '20210628', 'base_time' : '0600', 'nx' : '55', 'ny' : '127' }
```

```
response = requests.get(url, params=params)
```

```
print(response.content)
```

코드를 복사해서 기본 동작을 확인해 봅시다!

b) 요청 메시지 명세

항목명(영문)	항목명(국문)	항목크기	항목구분	샘플데이터	항목설명
serviceKey	인증키	100	1	인증키 (URL Encode)	공공데이터포털에서 발급받은 인증키
numOfRows	한 페이지 결과 수	4	1	10	한 페이지 결과 수 Default: 10
pageNo	페이지 번호	4	1	1	페이지 번호 Default: 1
dataType	응답자료형식	4	0	XML	요청자료형식(XML/JSON) Default: XML
base_date	발표일자	8	1	20210628	'21년 6월 28일 발표
base_time	발표시각	4	1	0600	06시 발표(정시단위) -매시각 10분 이후 호출
nx	예보지점 X 좌표	2	1	55	예보지점의 X 좌표값 *별첨 엑셀 자료 참조
ny	예보지점 Y 좌표	2	1	127	예보지점의 Y 좌표값 *별첨 엑셀 자료 참조

4. 공공데이터포털 기상청 API 이용하기

■ 샘플코드로 기본동작 확인하기

기상청 공공데이터 JSON 파싱 샘플 코드

샘플코드를 복사한 후 주요 파라미터만 수정해 보시다

```
import requests
```

```
url = 'http://apis.data.go.kr/1360000/VilageFcstInfoService_2.0/getUltraSrtNcst'
```

```
# 공공데이터포털에서 발급받은 본인의 서비스 키(Decoding) 입력
```

```
SERVICE_KEY = "RW3" = "
```

```
# 호출 기준 날짜 및 시간
```

```
base_date = '20260514' # 현재 날짜를 지정하세요
```

```
base_time = '1200' # 현재 시간을 지정하세요(40분 이전이면 한시간전 정시, 이후면 현재시간 정시 데이터)
```

```
# 특정 지역 격자 좌표 고정 선언 (예: 시흥시 정왕동)
```

```
FIXED_NX = 60
```

```
FIXED_NY = 127
```

```
params = {'serviceKey' : SERVICE_KEY, 'pageNo' : '1', 'numOfRows' : '1000', 'dataType' : 'JSON',  
          'base_date' : base_date, 'base_time' : base_time, 'nx' : FIXED_NX, 'ny' : FIXED_NY }
```

JSON으로

```
response = requests.get(url, params=params)
```

```
print(response.content)
```

[기상청 응답메시지의 JSON type의 원본 데이터]

```
b'{"response":{"header":{"resultCode":"00","resultMsg":"NORMAL_SERVICE"},"body":{"dataType":"JSON","items":  
{"item":[{"baseDate":"20260514","baseTime":"1200","category":"PTY","nx":60,"ny":127,"obsrValue":"0"}, {"baseDate":  
"20260514","baseTime":"1200","category":"REH","nx":60,"ny":127,"obsrValue":"35"}, {"baseDate":"2026051  
4","baseTime":"1200","category":"RN1","nx":60,"ny":127,"obsrValue":"0"}, {"baseDate":"20260514","baseTime":"12  
00","category":"T1H","nx":60,"ny":127,"obsrValue":"29.2"}, {"baseDate":"20260514","baseTime":"1200","categor  
y":"UUU","nx":60,"ny":127,"obsrValue":"-1.1"}, {"baseDate":"20260514","baseTime":"1200","category":"VEC","nx":  
60,"ny":127,"obsrValue":"108"}, {"baseDate":"20260514","baseTime":"1200","category":"VVV","nx":60,"ny":127,"ob  
srValue":"0.4"}, {"baseDate":"20260514","baseTime":"1200","category":"WSD","nx":60,"ny":127,"obsrValue":"1.  
2"}]}, "pageNo":1, "numOfRows":1000, "totalCount":8}}}'
```

4. 공공데이터포털 기상청 API 이용하기

■ 기상청 응답메시지의 데이터 분석(JSON)

기상청에서 받은 날씨데이터 중
3가지만 활용(기온, 습도, 강수형태)

```
{
  "response": {
    "header": {
      "resultCode": "00", "resultMsg": "NORMAL_SERVICE",
      "body": {
        "dataType": "JSON",
        "items": {
          "item": [
            {
              "baseDate": "20260514", "baseTime": "1200", "category": "PTY", "nx": 60, "ny": 127, "obsrValue": "0",
            },
            {
              "baseDate": "20260514", "baseTime": "1200", "category": "REH", "nx": 60, "ny": 127, "obsrValue": "35",
            },
            {
              "baseDate": "20260514", "baseTime": "1200", "category": "RN1", "nx": 60, "ny": 127, "obsrValue": "0",
            },
            {
              "baseDate": "20260514", "baseTime": "1200", "category": "T1H", "nx": 60, "ny": 127, "obsrValue": "29.2",
            },
            {
              "baseDate": "20260514", "baseTime": "1200", "category": "UUU", "nx": 60, "ny": 127, "obsrValue": "-1.1",
            },
            {
              "baseDate": "20260514", "baseTime": "1200", "category": "VEC", "nx": 60, "ny": 127, "obsrValue": "108",
            },
            {
              "baseDate": "20260514", "baseTime": "1200", "category": "VVV", "nx": 60, "ny": 127, "obsrValue": "0.4",
            },
            {
              "baseDate": "20260514", "baseTime": "1200", "category": "WSD", "nx": 60, "ny": 127, "obsrValue": "1.2"
            }
          ]
        }
      }
    }
  },
  "pageNo": 1, "numOfRows": 1000, "totalCount": 8
}
```

초단기실황	T1H	기온
	RN1	1시간 강수량
	UUU	동서바람성분
	VVV	남북바람성분
	REH	습도
	PTY	강수형태
	VEC	풍향
	WSD	풍속

5. Python requests 모듈을 이용해 기상청 날씨 정보 가공하기

■ 기상청 API로 기상데이터 받아서 처리하기

- API 호출 기준 날짜와 시간 지정하기

```
1 # 파이썬 requests 모듈을 이용한 기상청 데이터 가져오기
2 import requests, json
3 from datetime import datetime, timedelta # 날짜와 시간을 다루는 클래스
4
5 # 특정 지역 격자 좌표 지정 (지역: 시흥시 정왕동)
6 location = "시흥시 정왕동"
7 FIXED_NX = 56
8 FIXED_NY = 122
9
10 # 공공데이터포털에서 발급받은 본인의 서비스 키(Decoding)
11 SERVICE_KEY = 'RW[REDACTED]'
12
13 # 호출 기준 날짜 및 시간 계산하기
14 # 현재 시각을 기준으로 기상청 API 호출에 필요한 base_date와 base_time 계산하기
15 def get_datetime():
16     now = datetime.now()
17     # 초단기실황은 매시 40분 이후 안정적으로 생성되므로, 40분 기준으로 데이터 요청
18     if now.minute < 40: # 40분 전이면 '이전 시간 정시' 데이터 요청
19         api_time = now - timedelta(hours=1) # timedelta() 날짜나 시간의 차이를 계산
20         base_time = api_time.strftime("%H00") # strftime() 시간을 "문자열 포맷"으로 변환, 매시간 정시로 표현
21         base_date = api_time.strftime("%Y%m%d")
22     else: # 40분 이후면 '현재 시간 정시' 데이터 요청
23         base_time = now.strftime("%H00")
24         base_date = now.strftime("%Y%m%d")
25     return base_date, base_time
26
```

OpenAPI_kma_이름.py

5. Python requests 모듈을 이용해 기상청 날씨 정보 가공하기

■ url과 params 지정

- API 호출 URL과 파라미터 지정

```
26
27 # 호출 기준 날짜 및 시간 계산하는 함수 호출
28 base_date, base_time = get_datetime()
29
30 # 호출할 초단기실황 URL
31 url = 'http://apis.data.go.kr/1360000/VilageFcstInfoService_2.0/getUltraSrtNcst'
32 # 호출할 초단기실황 파라미터
33 params = {'serviceKey' : SERVICE_KEY,
34           'pageNo' : '1',
35           'numOfRows' : '1000',
36           'dataType' : 'JSON',
37           'base_date' : base_date,
38           'base_time' : base_time,
39           'nx' : FIXED_NX,
40           'ny' : FIXED_NY }
41
```

항목명(영문)	항목명(국문)	항목크기	항목구분	샘플데이터	항목설명
serviceKey	인증키	100	1	인증키 (URL Encode)	공공데이터포털에서 발급받은 인증키
numOfRows	한 페이지 결과 수	4	1	10	한 페이지 결과 수 Default: 10
pageNo	페이지 번호	4	1	1	페이지 번호 Default: 1
dataType	응답자료형식	4	0	XML	요청자료형식(XML/JSON) Default: XML
base_date	발표일자	8	1	20210628	'21년 6월 28일 발표
base_time	발표시각	4	1	0600	06시 발표(정시단위) -매시각 40분 이후 호출
nx	예보지점 X 좌표	2	1	55	예보지점의 X 좌표값 *별첨 엑셀 자료 참조
ny	예보지점 Y 좌표	2	1	127	예보지점의 Y 좌표값 *별첨 엑셀 자료 참조

5. Python requests 모듈을 이용해 기상청 날씨 정보 가공하기

■ 기상청으로 부터 응답데이터 받아오기

- requests.get() 이용해서 기상정보를 받아서 딕셔너리로 변환
- 딕셔너리 타입의 기상정보에서 필요한 정보(기온, 습도, 강수) 추출하기

```
42 # 기상청으로 응답메시지 받아오기
43 def forecast():
44     # GET 요청, url 주소와 파라미터
45     response = requests.get(url, params=params)
46     dict_data = response.json() # JSON --> Dict type으로 반환
47     #print("response_data:", dict_data)
48
49     weather_data = {}
50     # 날씨정보에서 기온, 습도, 강수정보 파싱
51     for item in dict_data['response']['body']['items']['item']:
52         if item['category'] == 'T1H': # 기온
53             weather_data['tmp'] = item['obsrValue']
54         if item['category'] == 'REH': # 습도
55             weather_data['hum'] = item['obsrValue']
56         # PTY (강수형태): 없음(0), 비(1), 빗방울(5)
57         if item['category'] == 'PTY':
58             weather_data['sky'] = item['obsrValue']
59     return weather_data
60
```

```
{
  "response": {
    "header": {
      "resultCode": "00",
      "resultMsg": "NORMAL_SERVICE"
    },
    "body": {
      "dataType": "JSON",
      "items": [
        {
          "item": [
            {
              "baseDate": "20260615",
              "baseTime": "1700",
              "category": "PTY",
              "nx": 55,
              "ny": 127,
              "obsrValue": "1"
            },
            {
              "baseDate": "20260615",
              "baseTime": "1700",
              "category": "REH",
              "nx": 55,
              "ny": 127,
              "obsrValue": "96"
            },
            {
              "baseDate": "20260615",
              "baseTime": "1700",
              "category": "RN1",
              "nx": 55,
              "ny": 127,
              "obsrValue": "0"
            },
            {
              "baseDate": "20260615",
              "baseTime": "1700",
              "category": "T1H",
              "nx": 55,
              "ny": 127,
              "obsrValue": "15.9"
            },
            {
              "baseDate": "20260615",
              "baseTime": "1700",
              "category": "UUU",
              "nx": 55,
              "ny": 127,
              "obsrValue": "1"
            },
            {
              "baseDate": "20260615",
              "baseTime": "1700",
              "category": "VEC",
              "nx": 55,
              "ny": 127,
              "obsrValue": "192"
            },
            {
              "baseDate": "20260615",
              "baseTime": "1700",
              "category": "VVV",
              "nx": 55,
              "ny": 127,
              "obsrValue": "4.7"
            },
            {
              "baseDate": "20260615",
              "baseTime": "1700",
              "category": "WSD",
              "nx": 55,
              "ny": 127,
              "obsrValue": "4.8"
            }
          ]
        }
      ]
    },
    "pageNo": 1,
    "numOfRows": 1000,
    "totalCount": 8
  }
}
```

5. Python requests 모듈을 이용해 기상청 날씨 정보 가공하기

■ 응답데이터에서 원하는 정보(기온, 습도, 날씨) 추출하기

- 받은 응답데이터에서 '기온', '습도', '날씨' 정보 추출하기

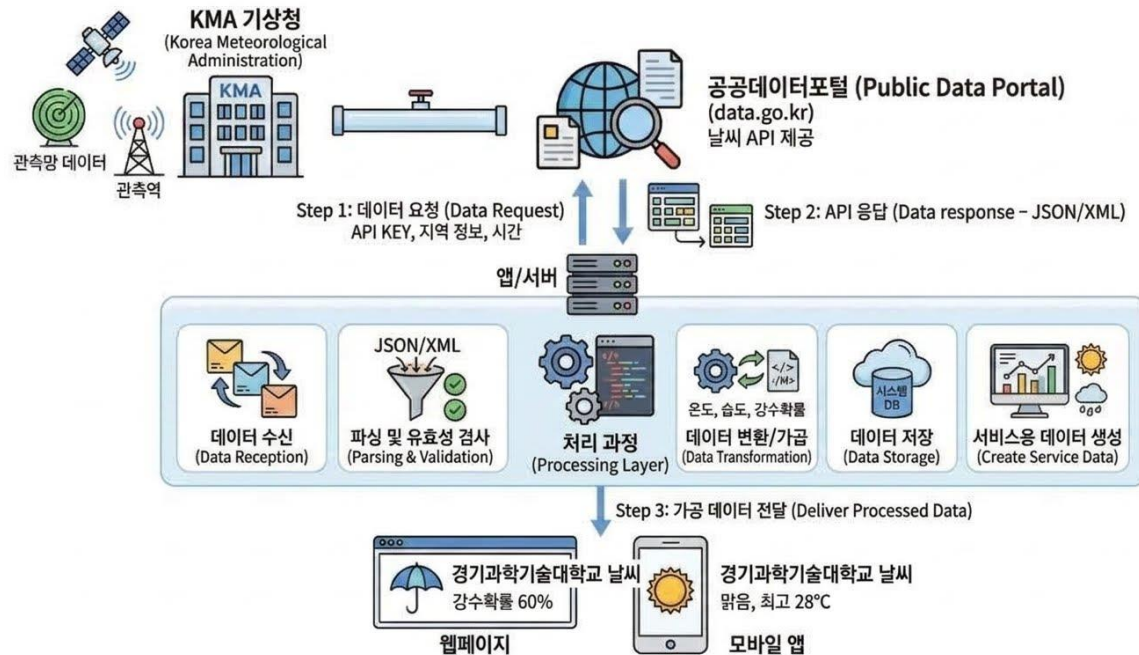
```
61 # 기상청데이터를 받아서 가공하기
62 def proc_weather():
63     dict_sky = forecast()
64     print(f"날씨데이터: {dict_sky}")
65     print(f"\n===== {base_date} {base_time} {location} 실시간 날씨 상황 =====")
66     if dict_sky['sky'] != None:
67         # 강수형태: 없음(0), 비(1), 비/눈(2), 눈(3), 빗방울(5), 빗방울눈날림(6), 눈날림(7)
68         if dict_sky['sky'] == '0':
69             print("날씨 : 맑음")
70         elif dict_sky['sky'] == '1' or '5':
71             print("날씨 : 비 또는 빗방울 떨어짐")
72         elif dict_sky['sky'] == '2' or '6':
73             print("날씨 : 진눈깨비")
74         elif dict_sky['sky'] == '3' or '7':
75             print("날씨 : 눈")
76     if dict_sky['tmp'] != None:
77         print("온도 :", dict_sky['tmp'] + "C")
78     if dict_sky['hum'] != None:
79         print("습도 :", dict_sky['hum'] + "%")
80
81 if __name__ == "__main__":
82     proc_weather()
```

- 강수형태(PTY) 코드 : (초단기) 없음(0), 비(1), 비/눈(2), 눈(3),
빗방울(5), 빗방울눈날림(6), 눈날림(7)

5. Python requests 모듈을 이용해 기상청 날씨 정보 가공하기

■ 출력결과

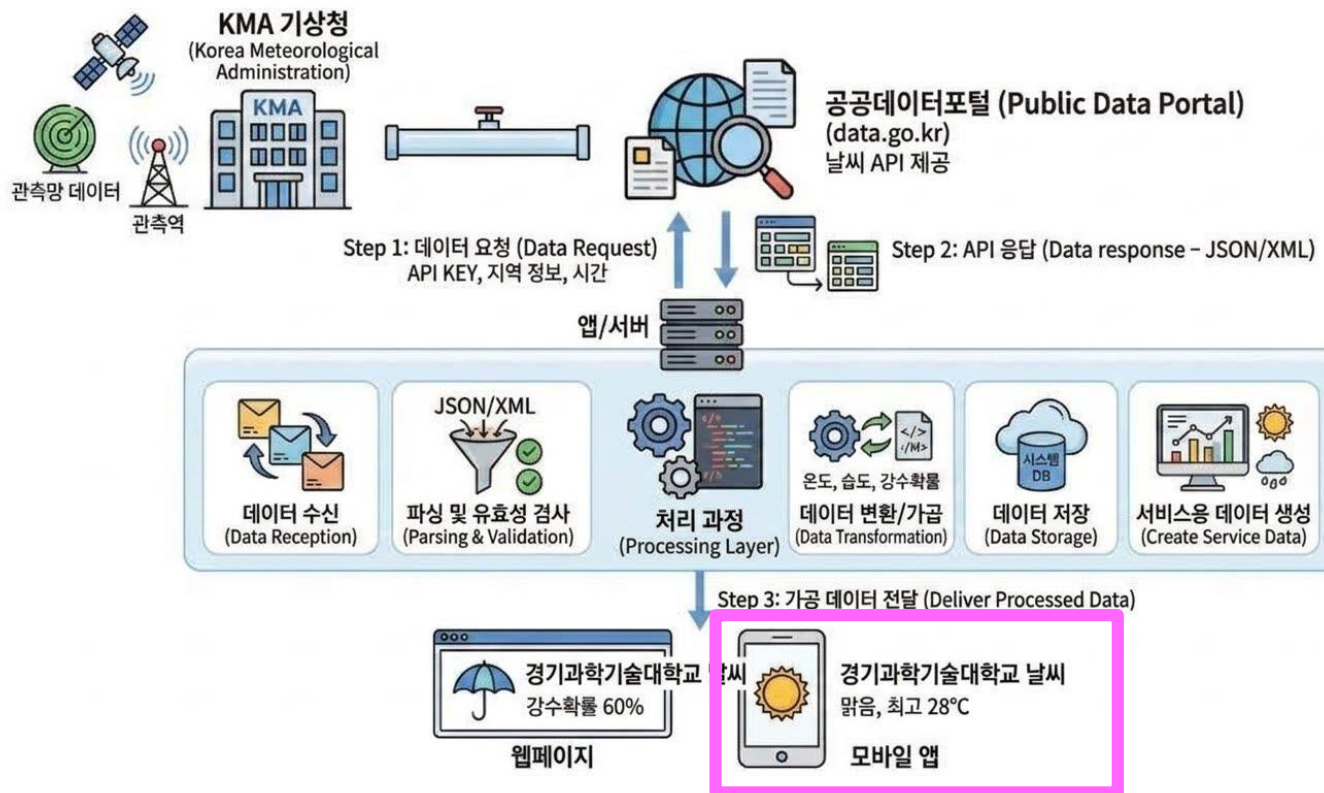
```
===== 20260514 1600 시흥시 정왕동 실시간 날씨 현황 =====  
날씨 : 맑음  
온도 : 28.2C  
습도 : 41%
```



6. OpenAPI를 이용한 기상청 날씨 앱 제작

■ 기상청 API로 날씨 정보 알아내는 앱 제작하기

- 기상청 Open API를 활용하여 날씨 정보를 받아오는 로직을 이해했다면 같은 방식으로 동작하는 앱을 제작해 봅시다



6. OpenAPI를 이용한 기상청 날씨 앱 제작

■ OpenAPI를 이용해 기상청으로부터 날씨정보 받아오는 앱 - 디자인

프로젝트 명: OpenAPI_weather

* Screen: 수평,수직- 가운데, 제목- 기상청 날씨정보 파싱(제작:홍길동)



	컴포넌트	이름	속성
①	레이블	레이블1_지역	글꼴굵게, 글꼴크기-20, 텍스트-지역
②	레이블	레이블1_요청날짜	글꼴굵게, 글꼴크기-20, 텍스트 - 요청날짜
③	레이블	레이블1_요청시간	글꼴굵게, 글꼴크기-20, 텍스트 - 요청시간
④	레이블	레이블1_기온	글꼴굵게, 글꼴크기-20, 텍스트-기온
⑤	버튼	버튼1_요청하기	배경색-주황, 글꼴굵게, 글꼴크기-20, 모양-직사각형, 텍스트-GET 요청
⑥	레이블	레이블1_응답메시지	글꼴굵게, 글꼴크기-15, 텍스트-응답메시지 *앱이 완성된 후 보이기여부 -해제
⑦	*웹	웹1	
⑧	*시계	시계1	

* 웹 : 연결 >> 웹

* 시계 : 센서 >> 시계

6. OpenAPI를 이용한 기상청 날씨 앱 제작

■ 코딩1) 코딩에 사용할 전역 변수 준비

- 기상청 서버로 부터 받은 데이터에서 원하는 기온 정보를 추출하는 코드 작성을 위해 데이터의 구조를 이해하기 위한 변수들 준비
: weather, weather_data, weather_temp
- API 요청에 필요한 APIkey 정보를 담은 변수 : APIkey
- API 요청에 필요한 날짜와 시간 정보를 담은 변수 : current_date, current_time

블록

공통 블록

- 제어
- 논리
- 수학
- 텍스트
- 리스트
- Dictionaries
- 색상
- 변수
- 함수

Screen1

레이블1_지역

뷰어

전역변수 만들기 APIkey 초기값 RV

전역변수 만들기 weather 초기값

전역변수 만들기 weather_data 초기값

전역변수 만들기 weather_temp 초기값

전역변수 만들기 current_date 초기값

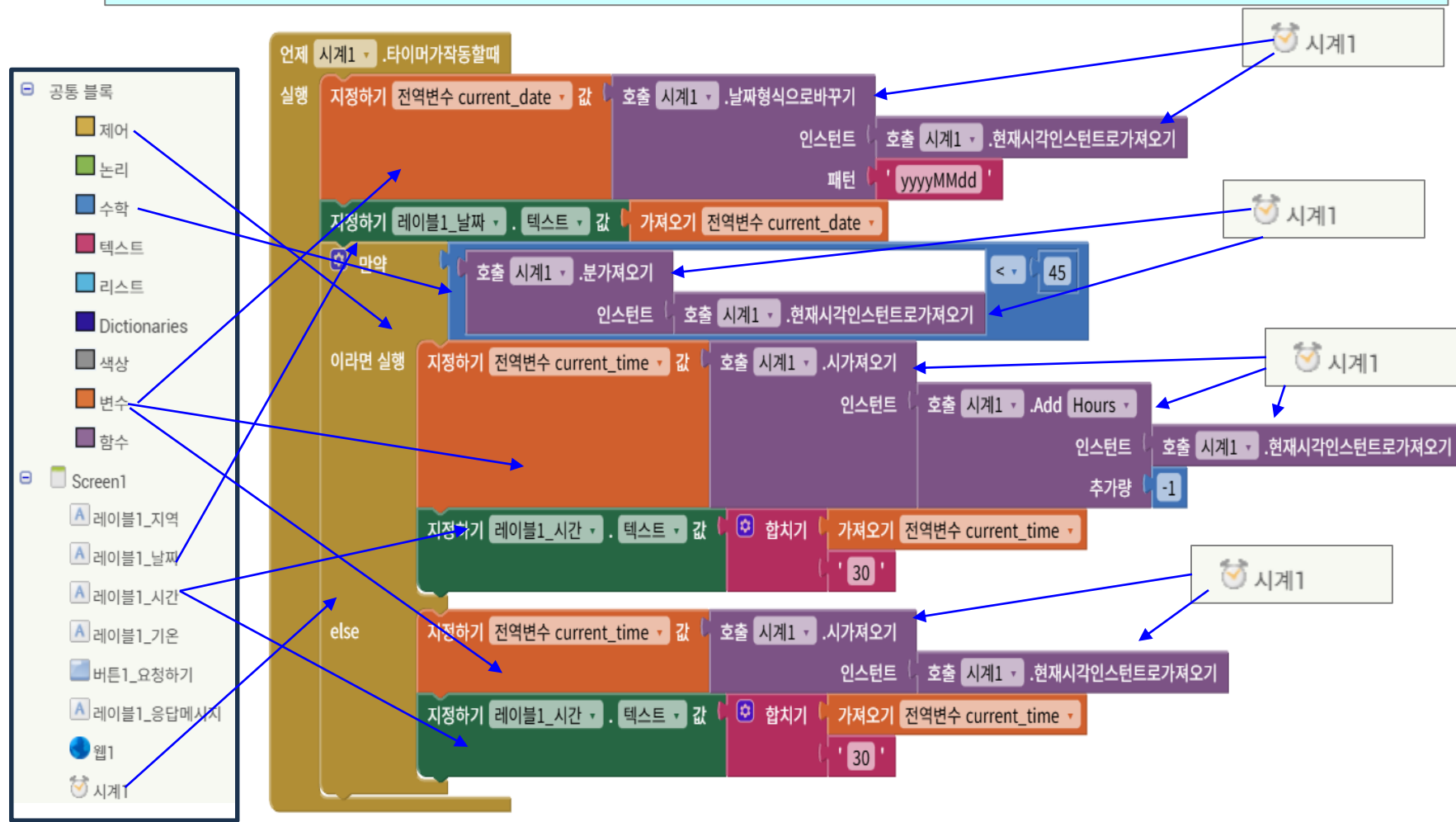
전역변수 만들기 current_time 초기값

공공데이터포털 기상청에서 받은 API key 정보
일반 인증키(Encoding)

6. OpenAPI를 이용한 기상청 날씨 앱 제작

■ 코딩2) GET 요청 시 쿼리스트링에 포함될 기준날짜, 기준시간 정보 만들기

- 기상청에 날씨정보 요청 시 쿼리스트링에 기준날짜와 시간정보를 전달해 주어야 함
 형식) 날짜형식 : 20260515, 시간형식:1300 (40분 기준으로)
 예) 2026년 5월 15일 14시 20분에 GET 요청 할 경우 (40분이전이면 이전시간 정각)
 - base_date:20250515, base_time: 1300



6. OpenAPI를 이용한 기상청 날씨 앱 제작

■ 코딩3) GET URL 작성하기

- GET 요청에 사용될 url 작성하기 (기본 url 주소 + ?쿼리스트링 파라미터)
- 기본 url = 'http://apis.data.go.kr/1360000/VilageFcstInfoService_2.0/getUltraSrtNcst'
- 요청 URL = 기본 URL?쿼리스트링 파라미터(key1=value1&key2=value2&key3=value3...)

쿼리스트링
파라미터

항목명(영문)	항목명(국문)
serviceKey	인증키
numOfRows	한 페이지 결과 수
pageNo	페이지 번호
dataType	응답자료형식
base_date	발표일자
base_time	발표시각
nx	예보지점 X 좌표
ny	예보지점 Y 좌표

get 요청

6. OpenAPI를 이용한 기상청 날씨 앱 제작

■ 코딩4) 1단계: 응답메시지 받아서 바디 출력하기

- 기상청 서버로부터 받은 응답메시지의 바디(JSON)를 딕셔너리 타입으로 **weather**에 저장 후 출력하기



- 기상청 서버로부터 받은 응답메시지 바디에 원하는 기온정보가 들어 있음

기상청 날씨정보 파싱

시흥시 정왕동
20250518
1230
기온
GET 요청

```
{
  "response": {
    "body": {
      "dataType": "JSON",
      "items": [
        {
          "item": {
            "baseDate": "20250518",
            "baseTime": "1300",
            "category": "PTY",
            "nx": 56,
            "ny": 122,
            "obsrValue": "0"
          },
          "baseDate": "20250518",
          "baseTime": "1300",
          "category": "REH",
          "nx": 56,
          "ny": 122,
          "obsrValue": "45"
        },
        {
          "baseDate": "20250518",
          "baseTime": "1300",
          "category": "RN1",
          "nx": 56,
          "ny": 122,
          "obsrValue": "0"
        },
        {
          "baseDate": "20250518",
          "baseTime": "1300",
          "category": "T1H",
          "nx": 56,
          "ny": 122,
          "obsrValue": "20"
        },
        {
          "baseDate": "20250518",
          "baseTime": "1300",
          "category": "UUU",
          "nx": 56,
          "ny": 122,
          "obsrValue": "2.3"
        },
        {
          "baseDate": "20250518",
          "baseTime": "1300",
          "category": "VEC",
          "nx": 56,
          "ny": 122,
          "obsrValue": "225"
        },
        {
          "baseDate": "20250518",
          "baseTime": "1300",
          "category": "VVV",
          "nx": 56,
          "ny": 122,
          "obsrValue": "2.3"
        },
        {
          "baseDate": "20250518",
          "baseTime": "1300",
          "category": "WSD",
          "nx": 56,
          "ny": 122,
          "obsrValue": "3.3"
        }
      ]
    },
    "numOfRows": 1000,
    "pageNo": 1,
    "totalCount": 8,
    "header": {
      "resultCode": "00",
      "resultMsg": "NORMAL SERVICE"
    }
  }
}
```

6. OpenAPI를 이용한 기상청 날씨 앱 제작

■ 코딩4) 2단계: 응답메시지 바디에서 item 정보만 추출하는 코드블럭

- 응답메시지 `weather`(딕셔너리 타입)에서 `item` 정보만 추출해 `weather_data`에 저장하기

{'response':

{'header': {'resultCode': '00', 'resultMsg': 'NORMAL_SERVICE'},

'body': {'dataType': 'JSON',

'items':

{'item':

```
[{'baseDate': '20250517', 'baseTime': '1700', 'category': 'PTY', 'nx': 55, 'ny': 127, 'obsrValue': '1'},  
{'baseDate': '20250517', 'baseTime': '1700', 'category': 'REH', 'nx': 55, 'ny': 127, 'obsrValue': '96'},  
{'baseDate': '20250517', 'baseTime': '1700', 'category': 'RN1', 'nx': 55, 'ny': 127, 'obsrValue': '0'},  
{'baseDate': '20250517', 'baseTime': '1700', 'category': 'T1H', 'nx': 55, 'ny': 127, 'obsrValue': '15.9'},  
{'baseDate': '20250517', 'baseTime': '1700', 'category': 'UUU', 'nx': 55, 'ny': 127, 'obsrValue': '1'},  
{'baseDate': '20250517', 'baseTime': '1700', 'category': 'VEC', 'nx': 55, 'ny': 127, 'obsrValue': '192'},  
{'baseDate': '20250517', 'baseTime': '1700', 'category': 'VVV', 'nx': 55, 'ny': 127, 'obsrValue': '4.7'},  
{'baseDate': '20250517', 'baseTime': '1700', 'category': 'WSD', 'nx': 55, 'ny': 127, 'obsrValue': '4.8'}],
```

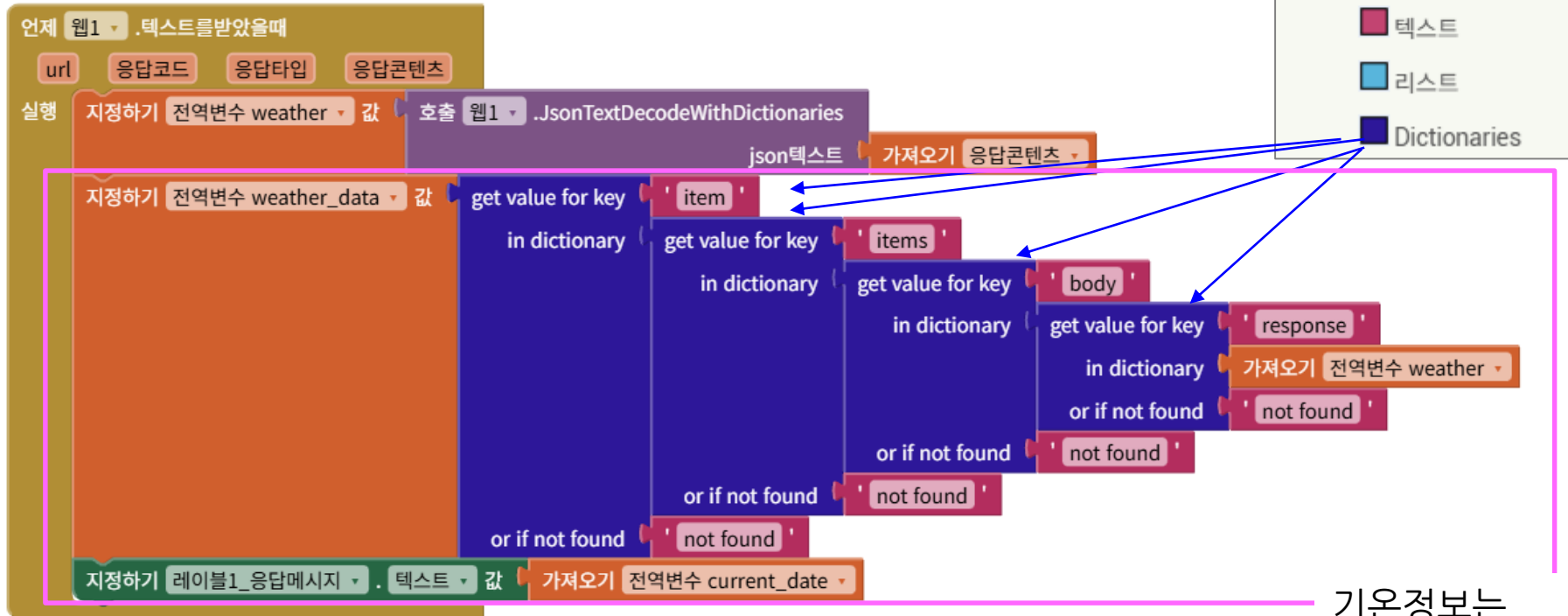
'pageNo': 1, 'numOfRows': 1000, 'totalCount': 8}}}



6. OpenAPI를 이용한 기상청 날씨 앱 제작

■ 코딩4) 2단계: item 정보만 추출하는 코드블럭 추가하기

- 응답메시지 `weather`(딕셔너리 타입)에서 **item 정보만** 추출해 `weather_data`에 저장하기 구현



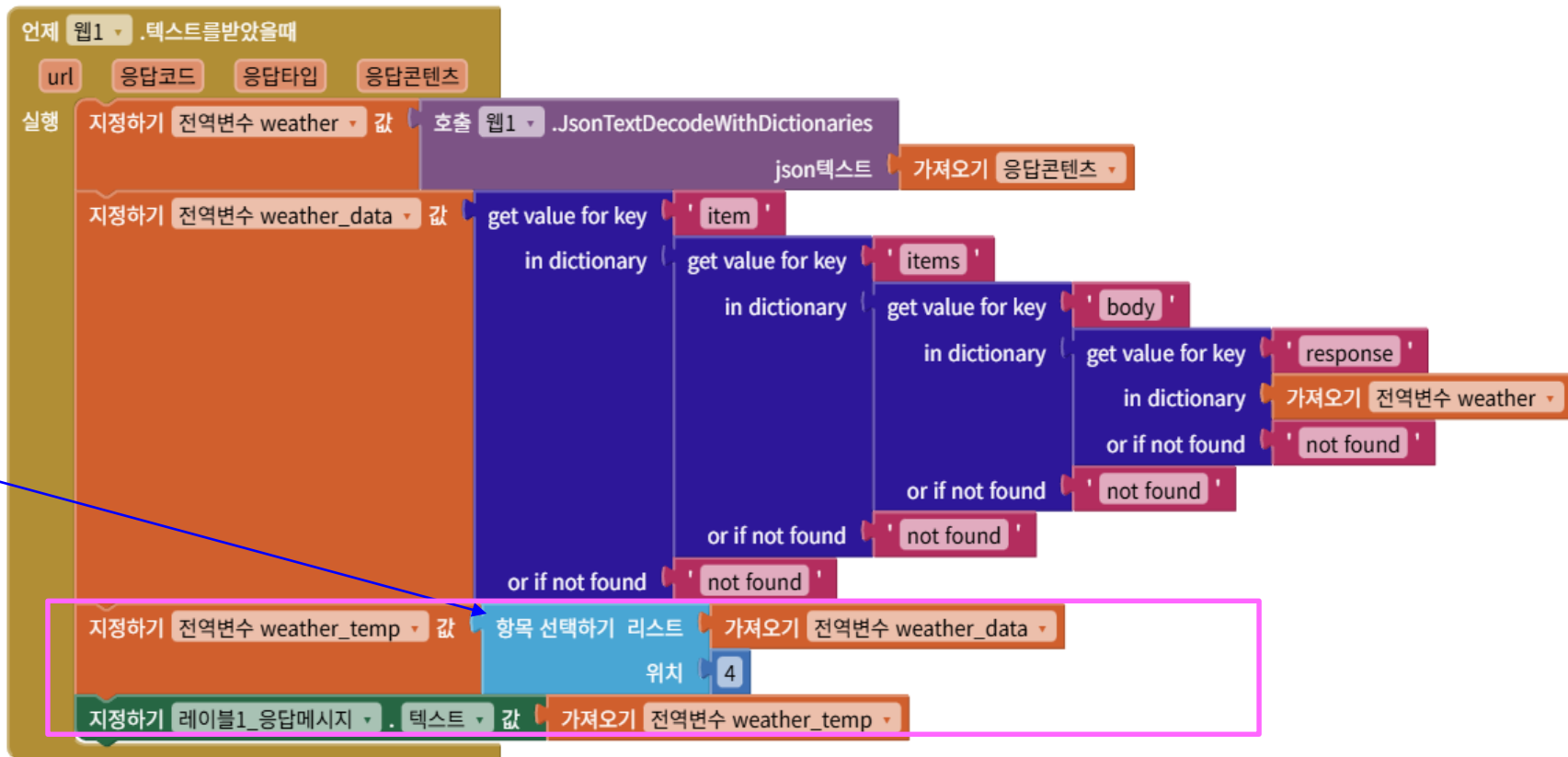
```
[{'baseDate': '20250517', 'baseTime': '1700', 'category': 'PTY', 'nx': 55, 'ny': 127, 'obsrValue': '1'},  
{'baseDate': '20250517', 'baseTime': '1700', 'category': 'REH', 'nx': 55, 'ny': 127, 'obsrValue': '96'},  
{'baseDate': '20250517', 'baseTime': '1700', 'category': 'RN1', 'nx': 55, 'ny': 127, 'obsrValue': '0'},  
{'baseDate': '20250517', 'baseTime': '1700', 'category': 'T1H', 'nx': 55, 'ny': 127, 'obsrValue': '15.9'},  
{'baseDate': '20250517', 'baseTime': '1700', 'category': 'UUU', 'nx': 55, 'ny': 127, 'obsrValue': '1'},  
{'baseDate': '20250517', 'baseTime': '1700', 'category': 'VEC', 'nx': 55, 'ny': 127, 'obsrValue': '192'},  
{'baseDate': '20250517', 'baseTime': '1700', 'category': 'VVV', 'nx': 55, 'ny': 127, 'obsrValue': '4.7'},  
{'baseDate': '20250517', 'baseTime': '1700', 'category': 'WSD', 'nx': 55, 'ny': 127, 'obsrValue': '4.8'}]
```

기온정보는
리스트 4번째
요소에 있음

6. OpenAPI를 이용한 기상청 날씨 앱 제작

■ 코딩4) 3단계: item 리스트에서 4번째 요소인 '기온'정보 추출하기

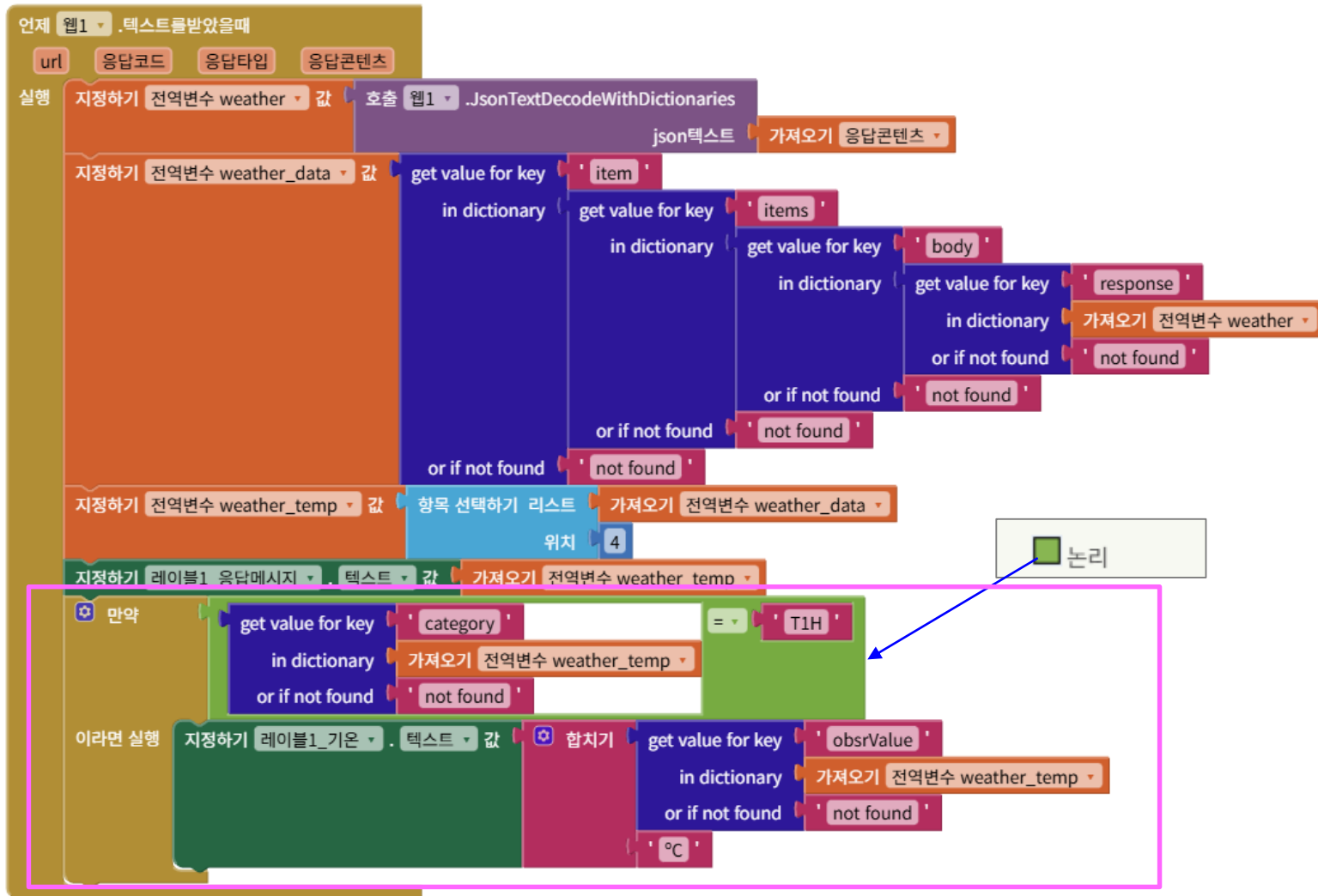
- item 리스트에서 기온정보가 포함된 4번째 요소만 `weather_temp`에 저장하기



{'baseDate': '20250517', 'baseTime': '1700', 'category': 'T1H', 'nx': 55, 'ny': 127, 'obsrValue': '15.9'}

6. OpenAPI를 이용한 기상청 날씨 앱 제작

- 코딩4) 4단계: 'category=T1H'에서 'obsrValue'의 값(기온) 추출하기



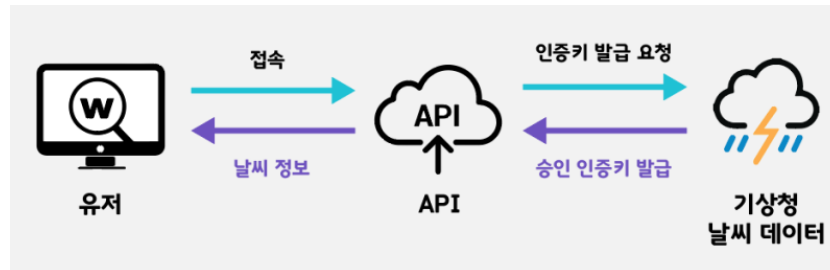
{'baseDate': '20250517', 'baseTime': '1700', 'category': 'T1H', 'nx': 55, 'ny': 127, 'obsrValue': '15.9'}

6. OpenAPI를 이용한 기상청 날씨 앱 제작

■ 앱 실행하기

기상청 날씨정보 파싱

시흥시 정왕동
20250518
1330
20.5°C
GET 요청



```
{'response':  
  {'header': {'resultCode': '00', 'resultMsg': 'NORMAL_SERVICE'},  
   'body': {'dataType': 'JSON',  
            'items':  
              {'item':  
                [{ 'baseDate': '20250517', 'baseTime': '1700', 'category': 'PTY', 'nx': 55, 'ny': 127, 'obsrValue': '1'},  
                  { 'baseDate': '20250517', 'baseTime': '1700', 'category': 'REH', 'nx': 55, 'ny': 127, 'obsrValue': '96'},  
                  { 'baseDate': '20250517', 'baseTime': '1700', 'category': 'RN1', 'nx': 55, 'ny': 127, 'obsrValue': '0'},  
                  { 'baseDate': '20250517', 'baseTime': '1700', 'category': 'T1H', 'nx': 55, 'ny': 127, 'obsrValue': '15.9'},  
                  { 'baseDate': '20250517', 'baseTime': '1700', 'category': 'UUU', 'nx': 55, 'ny': 127, 'obsrValue': '1'},  
                  { 'baseDate': '20250517', 'baseTime': '1700', 'category': 'VEC', 'nx': 55, 'ny': 127, 'obsrValue': '192'},  
                  { 'baseDate': '20250517', 'baseTime': '1700', 'category': 'VVV', 'nx': 55, 'ny': 127, 'obsrValue': '4.7'},  
                  { 'baseDate': '20250517', 'baseTime': '1700', 'category': 'WSD', 'nx': 55, 'ny': 127, 'obsrValue': '4.8'}]}}},  
  'pageNo': 1, 'numOfRows': 1000, 'totalCount': 8}}}
```


11주차 과제 안내

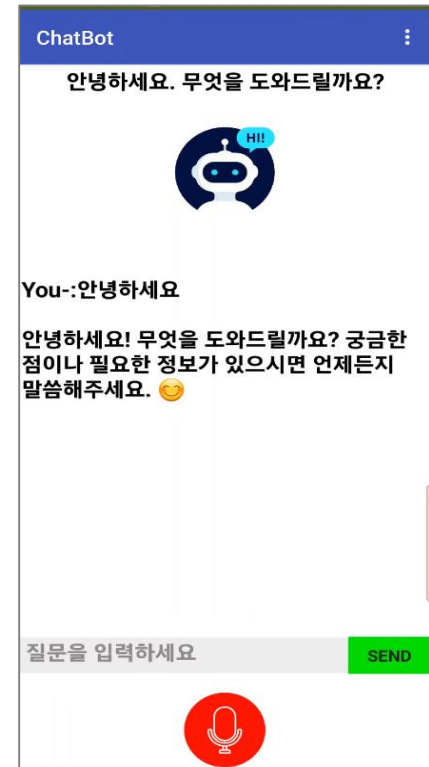
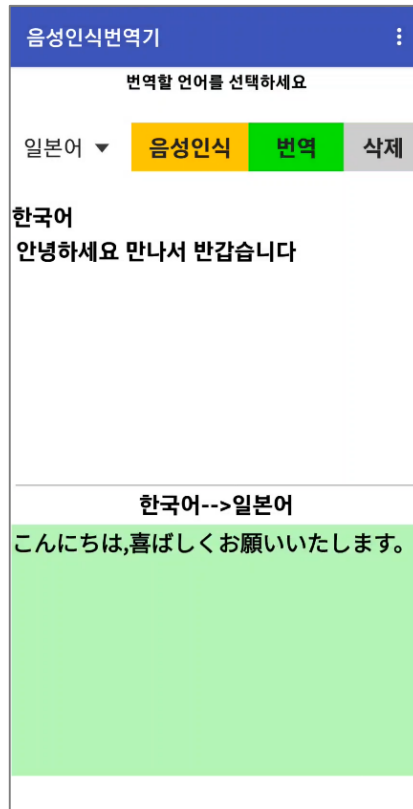
- 과제명: python requests 모듈을 이용해 기상청 날씨정보 가져오는 프로그램의 파이썬 파일을 제출해 주세요
 - 제출방법: 파이썬 파일을 PC에 저장 후 과제 제출란에 제출
 - 첨부파일: openAPI_kma_이름.py
 - 프로그램 상 오류 혹은 이해가 안 되는 부분은 comment 남겨 주세요.

```
===== 20260514 1600 시흥시 정왕동 실시간 날씨 현황 =====  
날씨 : 맑음  
온도 : 28.2C  
습도 : 41%
```

12주차 휴무일 영상수업 안내

■ 12주차는 임시이러닝 수업

- 학습방법: CTL 사이트 12주차 영상 수업 (임시 이러닝)
- 학습기간: 5/25(월) ~ 5/31(일)
- 주제: 앱 인벤터를 이용한 인공지능 앱 제작 실습
- 과제: 제작한 앱의 프로젝트 파일을 학습기간내 과제로 제출





Thank You
